UniBodget — Relazione di Progetto

Corso di Programmazione ad Oggetti — A.A. 2025/2026

Arianna Palmucci
arianna.palmucci@studio.unibo.it

Daniele Zappalà
daniele.zappala3@studio.unibo.it

Favour Osaretin Emuwahen
favour.emuwahen@studio.unibo.it

2 novembre 2026

Sommario

La presente relazione documenta l'analisi, la progettazione architetturale, lo sviluppo implementativo e la verifica del progetto software **UniBodget**, un'applicazione desktop modulare in Java finalizzata alla gestione completa delle finanze personali, della contabilità multi-asset, del monitoraggio di portafogli d'investimento e della conversione multivaluta in tempo reale.

Il documento è redatto in rigorosa aderenza alla struttura prescritta dalla *Meta-relazione per Programmazione ad Oggetti* (Prof. Danilo Pianini). In particolare, vengono analizzati e dettagliati i seguenti sottosistemi architetturali:

1. Il sottosistema del dominio valute (*fiat*, *crypto*, *stock*, *custom*) e il relativo motore polimorfico di conversione;
2. L'infrastruttura di integrazione con API di mercato esterne, supportata da una cache trasparente resiliente e meccanismi di fallback;
3. Il monitoraggio continuo dei tassi, con watchlist interattiva e sistema reattivo di allerte visive su soglie critiche;
4. Il modulo grafico dedicato al convertitore di valute (`CurrencyConverterModule`), comprendente il simulatore di cambio bancario con spread e commissioni fisse;
5. La gestione trasversale delle preferenze dell'utente (`SettingsController`), dei temi dinamici con iniezione CSS in tempo reale (`ThemeManager`) e della persistenza affidabile dello stato applicativo su file JSON strutturati.

Indice

1 Analisi **4**
- 1.1 Descrizione e requisiti . . . 4
- 1.2 Modello del dominio . . . 8

2 Design **13**
- 2.1 Architettura . . . 13
- 2.2 Design Dettagliato (Arianna Palmucci) . . . 14
  - 2.2.1 Sistema Polimorfico delle Valute e Motore di Conversione . . . 14
  - 2.2.2 Architettura dei Provider di Cambio con Caching Resiliente . . . 18
  - 2.2.3 Monitoraggio dei Tassi, Allerte di Mercato e Watchlist Reattiva . . . 21
  - 2.2.4 Gestione delle Preferenze Utente, Temi Dinamici e Memento dello Stato . . . 22
  - 2.2.5 Composizione, Accessibilità e Ottimizzazione della View . . . 25
  - 2.2.6 Risoluzione Resiliente del File System e Autoconsistenza dei Dati (AppDataPathResolver) . . . 27
  - 2.2.7 Normalizzazione del Parsing e Validazione dell'Input Monetario (MonetaryInputParser) . . . 29
- 2.3 Design Dettagliato (Favour Osaretin Emuwahen) . . . 31
  - 2.3.1 Coordinamento e Disaccoppiamento della Dashboard (Facade e Passive View) . . . 31
  - 2.3.2 Costruzione Fluente e Validazione delle Transazioni (Builder e Factory Pattern) . . . 33
  - 2.3.3 Monitoraggio Dinamico e Strategie di Allerta del Budget (Strategy Pattern) . . . 34
  - 2.3.4 Algoritmo di Analisi Comparativa Mensile (Template Method Pattern) 36
  - 2.3.5 Filtraggio Composito ed Estendibile della Cronologia (Specification Pattern) . . . 37
- 2.4 Design Dettagliato (Daniele Zappalà) . . . 40
  - 2.4.1 Ordini con Sorgenti di Pagamento Diverse . . . 40
  - 2.4.2 Saldo e Posizioni Calcolati dallo Storico . . . 41
  - 2.4.3 Eventi, Tempo dell'Applicazione e Salvataggi Automatici . . . 41
  - 2.4.4 Finestra Principale e Barra Laterale Condivisa . . . 42

3 Sviluppo **44**
- 3.1 Testing Automatizzato . . . 44
- 3.2 Note di Sviluppo . . . 46
  - 3.2.1 Favour Osaretin Emuwahen . . . 46
  - 3.2.2 Arianna Palmucci . . . 48
  - 3.2.3 Daniele Zappalà . . . 49

4 Commenti finali **51**
4.1 Autovalutazione e lavori futuri 51
4.1.1 Favour Osaretin Emuwahen 51
4.1.2 Arianna Palmucci 52
4.1.3 Daniele Zappalà 54
4.1.4 Sviluppi Futuri Comuni 55
4.2 Difficoltà Incontrate e Commenti per i Docenti 55

A Guida Utente **56**

B Esercitazioni di Laboratorio **62**

Capitolo 1

Analisi

1.1 Descrizione e requisiti

UniBodget è un'applicazione desktop per la gestione delle finanze personali. Il suo scopo è riunire in un unico posto il denaro usato tutti i giorni (contanti, conti correnti, carte) e il patrimonio investito (azioni e criptovalute), anche quando questi sono espressi in valute diverse.

Oggi una persona tiene spesso i propri soldi su più canali: uno o più conti in euro, magari un conto in un'altra valuta, un broker per le azioni e un exchange per le criptovalute. Ognuno di questi ha la propria interfaccia e la propria unità di misura, per cui rispondere a domande semplici come "quanto ho speso questo mese?" o "i miei investimenti stanno guadagnando o perdendo?" richiede di consultare più strumenti e di fare conversioni a mano. UniBodget vuole rispondere a queste domande da un'unica applicazione, che lavora sul computer dell'utente senza richiedere l'accesso ai suoi conti bancari.

Glossario

Valuta (o unità di valore) qualunque cosa in cui si può esprimere una quantità di denaro o di patrimonio. Si distinguono tre tipi: valute *fiat* (euro, dollaro, ...), *criptovalute* (bitcoin, ether, ...) e *azioni* quotate in borsa, identificate dal loro simbolo (es. AAPL).

Stablecoin criptovaluta il cui valore è agganciato a una valuta fiat (es. USDT, circa 1 dollaro). Può essere usata come "liquidità" all'interno di un conto di investimento.

Importo una quantità associata a una valuta, ad esempio 12,50 EUR oppure 0,003 BTC.

Conto contenitore di denaro o di patrimonio con un nome e una *valuta di riferimento*, in cui vengono espressi i suoi totali. I conti sono di due tipi:

- *conto di liquidità*: denaro per le spese di tutti i giorni (es. conto corrente, contanti); il suo saldo è la somma dei movimenti registrati;
- *conto di investimento*: contiene azioni e criptovalute; il suo valore dipende dai prezzi di mercato del momento.

Movimento (o transazione) registrazione di un'operazione su un conto: un'entrata, una spesa, un trasferimento, un prestito a un amico, oppure un acquisto o una vendita di un titolo.

Categoria etichetta che classifica un movimento (es. “Spesa alimentare”, “Stipendio”), con un colore e un tipo (entrata, uscita, trasferimento, prestito).

Posizione quanto si possiede di un certo titolo in un conto di investimento, con il suo *prezzo medio di carico*, cioè il prezzo medio pagato per acquistarlo.

Costo di carico quanto è stato speso in totale per acquistare le posizioni ancora aperte.

Profitto/perdita (P/L) differenza tra valore e costo. È *non realizzato* finché il titolo è ancora posseduto, *realizzato* quando viene venduto.

Spread maggiorazione percentuale che una banca applica al tasso di cambio di mercato quando converte denaro da una valuta a un’altra, spesso insieme a una commissione fissa.

Budget limite di spesa mensile fissato dall’utente per un conto di liquidità, con una soglia di avviso espressa in percentuale del limite.

Cruscotto (dashboard) schermata di riepilogo di un conto di liquidità: saldo, budget, spese per categoria, prestiti, confronti con il mese precedente e movimenti recenti.

Prestito ad amici denaro dato a un amico (o restituito da lui) che l’utente vuole tracciare, per sapere in ogni momento quanto deve ancora ricevere.

Confronto mensile (insight) indicatore che confronta un valore del mese corrente (spese, entrate, risparmi) con quello del mese precedente, indicando se l’andamento è positivo, negativo o stabile.

Watchlist elenco di coppie di valute che l’utente vuole tenere sotto controllo.

Requisiti funzionali

Conti e movimenti

- L’utente può creare, rinominare ed eliminare più conti di liquidità, ognuno con la propria valuta di riferimento.
- L’utente può registrare entrate, spese, trasferimenti e prestiti fatti a (o ricevuti da) amici, indicando importo, data, categoria e una descrizione.
- Un movimento registrato per errore può essere corretto o eliminato.
- Oltre alle categorie predefinite, l’utente può creare categorie personalizzate con nome, tipo e colore.
- Lo storico dei movimenti si può cercare per parola chiave e filtrare per categoria, tipo e intervallo di date, ordinandolo in modi diversi.
- Lo storico di un conto si può esportare in un file leggibile da un foglio di calcolo.

Cruscotto, budget e prestiti

- L'utente sceglie il conto di liquidità da consultare; il cruscotto ne mostra il saldo, lo stato del budget, la ripartizione delle spese, i prestiti aperti, i confronti mensili e gli ultimi movimenti, e si aggiorna subito a ogni modifica.
- Per ogni conto di liquidità l'utente può fissare un budget mensile e una soglia di avviso (es. l'80% del limite). L'applicazione indica quanta parte del budget è stata usata e in quale stato si trova la spesa del mese: *nella norma*, *in avviso* (soglia superata) o *critica* (limite raggiunto o superato).
- L'applicazione mostra come si ripartiscono le spese del mese tra le categorie, sia in percentuale sia come elenco delle categorie in cui si spende di più.
- L'applicazione confronta spese, entrate e risparmi del mese corrente con quelli del mese precedente, mostrando la differenza in valore e in percentuale e se l'andamento è positivo o negativo.
- L'utente può registrare un prestito fatto a un amico e, in seguito, le sue restituzioni (anche parziali). Per ogni amico l'applicazione riassume quanto è stato prestato, quanto è stato restituito e quanto resta da ricevere.

Investimenti

- L'utente può creare, rinominare ed eliminare più conti di investimento.
- L'utente può registrare acquisti e vendite di azioni e criptovalute, indicando quantità, prezzo unitario e commissioni, e può trasferire un titolo da un conto di investimento a un altro.
- Un acquisto può essere pagato con un conto di liquidità, con una stablecoin posseduta nello stesso conto, oppure con fondi esterni non gestiti dall'applicazione. Allo stesso modo, il ricavato di una vendita può essere accreditato su un conto di liquidità o in stablecoin.
- Un ordine che non può essere eseguito (fondi insufficienti, quantità da vendere maggiore di quella posseduta, ...) deve essere rifiutato, spiegando il motivo all'utente, senza modificare nessun conto.
- Per ogni posizione l'applicazione mostra quantità, prezzo medio di carico, valore di mercato attuale e profitto/perdita non realizzato.
- Per ogni conto di investimento l'applicazione mostra il valore totale, il costo di carico, il profitto/perdita complessivo (realizzato e non realizzato) in valore e in percentuale, e i titoli con il rendimento migliore e peggiore.
- L'applicazione mostra come il conto è ripartito tra i titoli, sia per valore sia per quantità.
- L'applicazione mostra l'andamento nel tempo del profitto/perdita del conto: il valore viene registrato dopo ogni operazione e periodicamente mentre l'applicazione è aperta.
- Lo storico delle operazioni di un conto di investimento si può esportare in un file leggibile da un foglio di calcolo.

Conversione di valute e mercati

- L'utente può convertire un importo tra due valute qualsiasi tra quelle supportate (fiat, criptovalute e azioni), al tasso di mercato corrente.
- L'utente può simulare il cambio di denaro presso diverse banche, vedendo spread, commissione fissa e importo che riceverebbe davvero.
- L'utente può vedere in un grafico l'andamento del tasso di cambio di una coppia di valute in un intervallo di date scelto.
- L'utente può salvare coppie di valute preferite in una watchlist, per vederne subito il tasso.
- L'utente può impostare un avviso su una coppia di valute: quando il tasso supera la soglia impostata, l'applicazione lo segnala.

Preferenze

- L'utente sceglie una valuta di riferimento globale, in cui vengono mostrati i totali che riuniscono più conti.
- L'utente può passare tra tema chiaro e tema scuro e cambiare il carattere del testo; le modifiche si vedono subito, senza riavviare l'applicazione.
- L'applicazione ricorda dimensione e posizione della finestra tra un avvio e l'altro.
- L'utente può salvare la configurazione corrente e tornare in seguito a una configurazione salvata in precedenza.

Dati

- Ogni modifica (conti, movimenti, categorie, preferenze) viene salvata automaticamente, senza un comando esplicito dell'utente, e ritrovata al successivo avvio.
- Al primo avvio, senza dati salvati, l'applicazione parte da uno stato vuoto ma funzionante.

Requisiti non funzionali

- **Precisione**: i calcoli sugli importi devono essere esatti, senza gli errori di arrotondamento tipici dei numeri in virgola mobile. Ogni valuta va mostrata con il numero di decimali adatto (2 per l'euro, fino a 8 per le criptovalute).
- **Coerenza dei dati**: saldi e statistiche devono essere sempre coerenti con lo storico dei movimenti, anche dopo una correzione o una cancellazione. Un'operazione che coinvolge due conti (ad esempio un acquisto pagato con un conto di liquidità) deve modificarli entrambi oppure nessuno.
- **Robustezza verso i servizi esterni**: i prezzi di mercato arrivano da servizi pubblici e gratuiti, che possono essere lenti, limitare il numero di richieste o non essere raggiungibili. L'applicazione deve continuare a funzionare anche in questi casi (ad esempio mostrando l'ultimo prezzo noto) e non deve mai perdere dati dell'utente. Non è richiesto che i prezzi siano in tempo reale: un ritardo dell'ordine di un'ora è accettabile.

- **Privacy**: tutti i dati restano sul computer dell'utente; non servono registrazioni né credenziali bancarie.
- **Portabilità**: l'applicazione deve funzionare su Windows, macOS e Linux senza modifiche.
- **Estendibilità**: deve essere possibile aggiungere nuovi tipi di valuta o di mercato, nuove categorie e nuove fonti di prezzi senza riscrivere le parti esistenti.

1.2 Modello del dominio

Il modello del dominio è riassunto in Figura 1.1 e Figura 1.2, dove le entità sono suddivise tra la gestione dei conti, storico e budget (Parte 1) e l'ecosistema di mercato, tassi e valute (Parte 2). Di seguito le aree sono descritte una alla volta.

Valute e importi Ogni quantità di denaro o di patrimonio è un **importo** (`Asset`): un numero espresso in una **valuta** (`CurrencyUnit`). Le valute sono di tipi diversi, valute fiat (`FiatCurrency`), criptovalute (`CryptoCurrency`) e azioni (`StockMarketCurrency`), più eventuali valute personalizzate (`Currency`); per l'importo il tipo non fa differenza: 10 EUR, 0,5 BTC e 3 AAPL sono tutti importi.

Conti e movimenti Il dominio ruota attorno al concetto di **conto** (`Wallet`). Un conto ha un nome, una valuta di riferimento e uno **storico** (`Historical`) che registra tutti i suoi **movimenti** (`Transaction`). Il saldo non è un'informazione a sé, ma si ricava sempre dallo storico. Ogni movimento ha una data, una descrizione, un importo e una **categoria** (`Category`) con nome, colore e tipo (entrata, spesa, trasferimento, prestito). Esistono due tipi di conto, e ognuno accetta un solo tipo di movimento:

- il **conto di liquidità** (`CashAccount`) contiene movimenti di denaro (`CashTransaction`); un movimento può riferirsi a un amico, quando rappresenta un prestito o una sua restituzione;
- il **conto di investimento** (`InvestmentAccount`) contiene acquisti, vendite e trasferimenti di titoli (`InvestmentTransaction`), che oltre all'importo registrano il prezzo unitario e la commissione.

Cruscotto Ogni conto di liquidità ha un **budget** mensile (`BudgetSettings`), cioè un limite di spesa e una soglia di avviso. Confrontando la spesa del mese con il budget si ottiene lo **stato del budget** (`BudgetStatus`): nella norma, in avviso o critico (`BudgetMonitor`). Dai movimenti di liquidità si ricavano anche i **riepiloghi dei prestiti** (`FriendLoanSummary`), uno per amico, con quanto è stato prestato, quanto restituito e il saldo residuo (`FriendLoanSummary`), e i **confronti mensili** (`WalletInsight`), che dicono come spese, entrate e risparmi sono cambiati rispetto al mese precedente (`WalletInsight`). Queste informazioni non vengono memorizzate: si calcolano ogni volta dai movimenti.

Investimenti Dai movimenti di un conto di investimento si ricavano le **posizioni** (`Position`), una per ogni titolo posseduto, con quantità, prezzo medio di carico e profitto/perdita. Il valore delle posizioni dipende dal prezzo di mercato del momento, che il conto ottiene da un **convertitore** (`CurrencyConverter`) capace di esprimere un importo in un'altra valuta. Per seguire l'andamento del conto nel tempo se ne registrano delle **istantanee** (`BalanceSnapshot`) con il profitto/perdita e il costo di carico in un certo momento.

Mercato e preferenze Gli **avvisi** (`CurrencyAlert`) legano una coppia di valute a una soglia e indicano se il tasso di cambio la supera. Le **preferenze** dell'utente (`Settings`) contengono la valuta di riferimento globale, il tema grafico e la cronologia delle configurazioni salvate. Restano fuori dallo schema tre concetti semplici e indipendenti dal resto: la **banca** (`Bank`), descritta da nome, spread percentuale e commissione fissa; la **watchlist** (`WatchList`), un insieme di coppie di codici di valuta (`WatchlistPair`); le fonti dei prezzi di mercato, che servono al convertitore ma non fanno parte del problema.

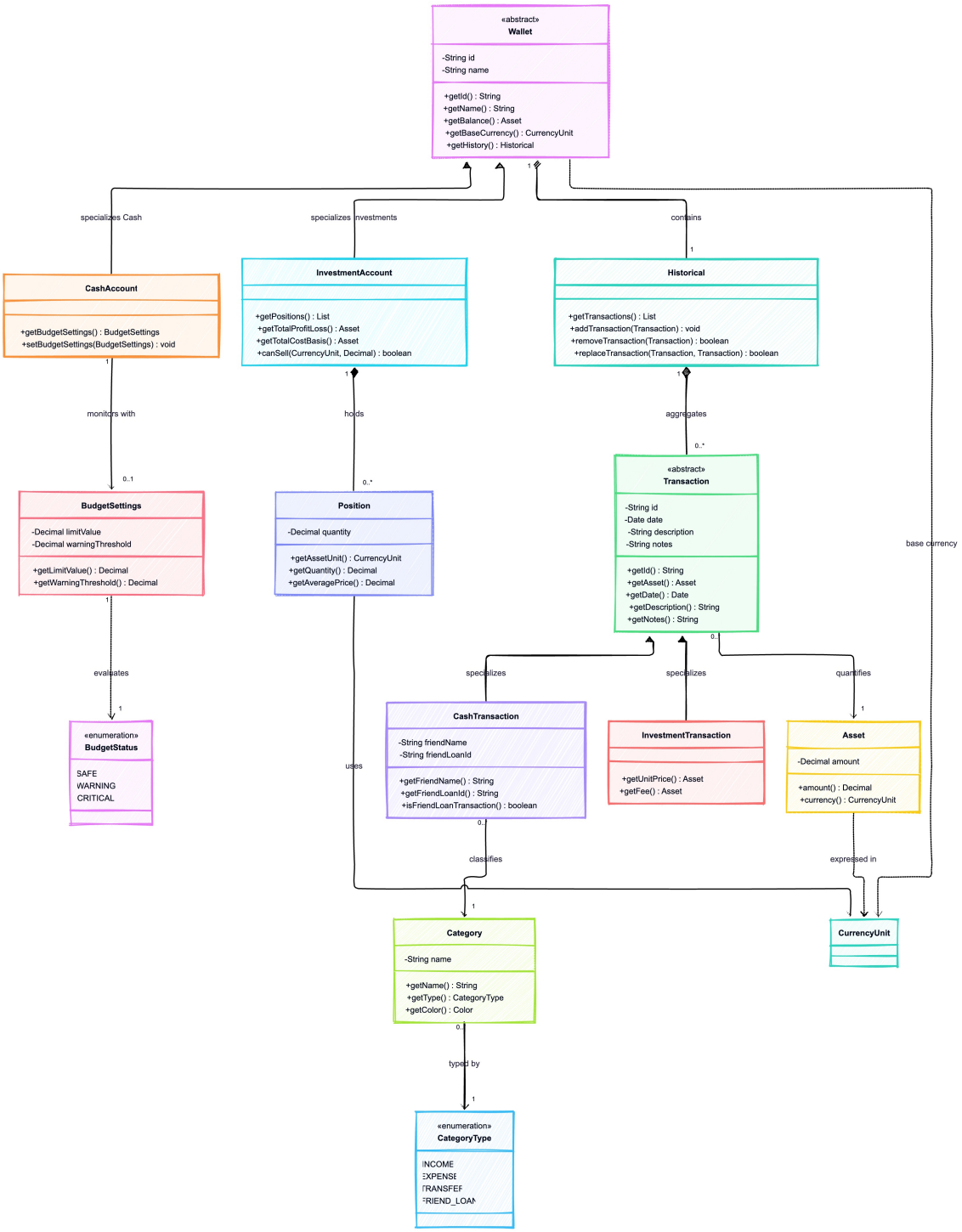

Figura 1.1: Schema UML del dominio (Parte 1): conti, storico transazioni, posizioni e gestione del budget.

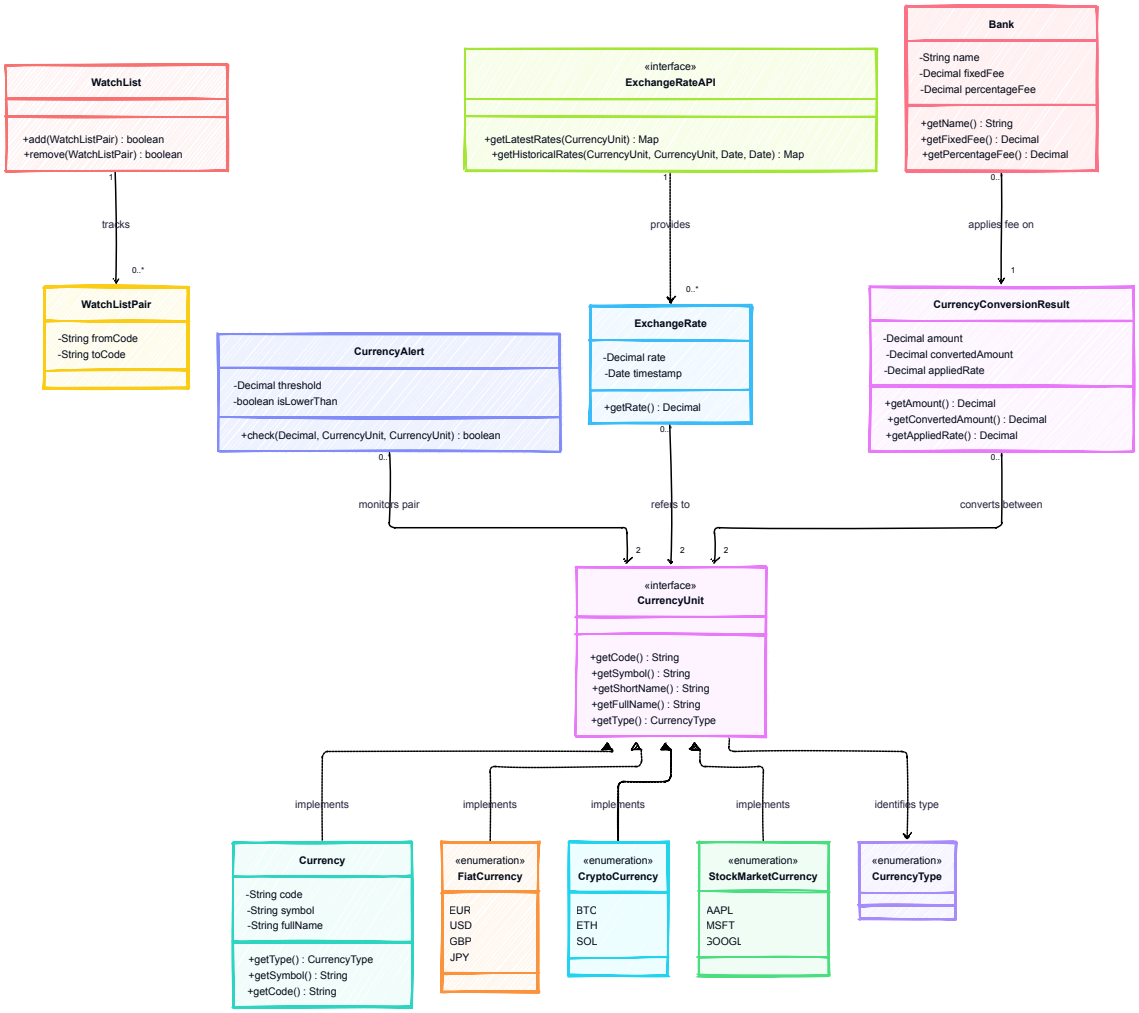

Figura 1.2: Schema UML del dominio (Parte 2): mercati, tassi di cambio, watchlist, banche e conversioni valutarie.

Aspetti critici

Alcuni aspetti del problema sono particolarmente impegnativi.

- **Valute eterogenee**. Euro, bitcoin e azioni hanno precisioni molto diverse e i loro prezzi provengono da fonti diverse. Non esiste un tasso diretto per ogni coppia di valute (ad esempio tra un'azione e una criptovaluta), e i tassi cambiano continuamente.
- **Valore degli investimenti**. Il saldo di un conto di liquidità dipende solo dalla sua storia; il valore di un conto di investimento dipende anche dal prezzo di mercato del momento. Con più acquisti a prezzi diversi e vendite parziali, il prezzo medio di carico e la distinzione tra profitto realizzato e non realizzato richiedono regole di calcolo precise.
- **Operazioni su più conti**. Un acquisto pagato con un conto di liquidità modifica due conti di tipo diverso, eventualmente in valute diverse; il pagamento può anche avvenire con una stablecoin dello stesso conto.
- **Correzioni nel passato**. Correggere o eliminare un movimento già registrato cambia tutti i saldi e le statistiche che ne dipendono.
- **Andamento nel tempo**. Per mostrare l'andamento di un conto di investimento il suo valore va registrato nel tempo, anche quando l'utente non fa nessuna operazione.
- **Fonti di prezzo esterne**. I servizi di quotazione gratuiti hanno limiti sul numero di richieste e possono non rispondere, mentre l'applicazione deve restare utilizzabile.

Alcune funzionalità sono state lasciate fuori da questa versione: l'importazione automatica dei movimenti dalle banche, le quotazioni in tempo reale, il calcolo delle tasse sui guadagni e la gestione di più utenti. Le valute supportate sono un elenco predefinito, che può essere esteso ma non dall'interfaccia dell'applicazione.

Capitolo 2

Design

In questo capitolo vengono descritte la struttura architetturale complessiva di UniBodget e le strategie di progettazione dettagliate adottate da **Arianna Palmucci** per la risoluzione dei problemi cardine a lei assegnati.

2.1 Architettura

Il sistema adotta il pattern architetturale **Model-View-Controller (MVC)** integrato con una struttura reattiva basata su un bus di messaggi disaccoppiato (`MessageBus`), come sintetizzato nello schema di Figura 2.1. Tale impostazione garantisce il completo isolamento della logica applicativa rispetto al toolkit grafico (JavaFX 21).

Ruoli dei Componenti Architetturali

- **Model:** Incapsula lo stato persistente dell'applicazione, le regole di business contabili, gli algoritmi di conversione e i meccanismi di serializzazione. Il modello è puro: non contiene alcuna dipendenza verso librerie grafiche (`javafx.*`) ed è completamente testabile in ambienti headless.
- **Controller:** Intercetta i comandi dell'utente inviati dalla vista, coordina le invocazioni verso i servizi del modello e trasforma le entità di business in strutture dati di presentazione (view models e records immutabili) consumabili dalla GUI.
- **View:** Costruita interamente in JavaFX attraverso una combinazione di file FXML e viste programmatiche modulari. Si occupa unicamente del rendering grafico e dell'ascolto delle interazioni utente, delegando ogni logica decisionale al rispettivo controller.

Disaccoppiamento View-Controller

Il disaccoppiamento è garantito dall'adozione del principio di inversione delle dipendenze:

- I controller non detengono riferimenti a classi di layout o nodi visivi specifici (non manipolano `TextField`, `TableView` o controlli FXML), ma operano tramite interfacce di callback (`Consumer`, `Runnable`, `Supplier`).
- Nel modulo di conversione (`CurrencyConverterModule`), l'intera vista è esposta verso la shell principale come un generico nodo radice `Parent`, rendendo la composizione dell'interfaccia totalmente agnostica rispetto ai widget interni.
- La sostituzione completa del framework grafico (ad esempio verso Swing o una web UI) richiederebbe unicamente la riscrittura del layer View, lasciando immutati sia il Controller sia l'intero Model.

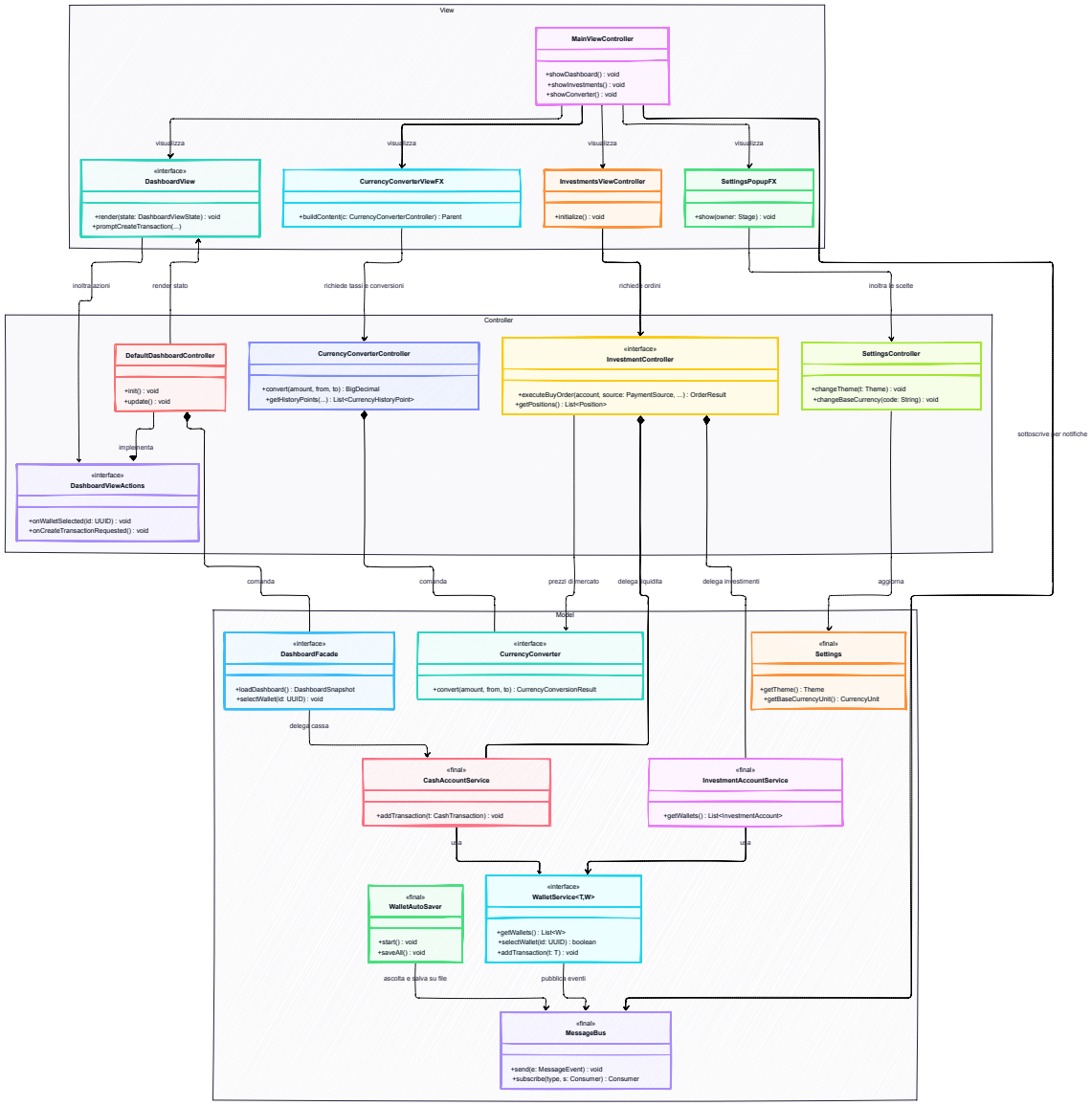

Figura 2.1: Rappresentazione ad alto livello dell'architettura MVC e del MessageBus disaccoppiato.

2.2 Design Dettagliato (Arianna Palmucci)

In questa sezione vengono dettagliati i cinque principali problemi di progettazione affrontati e risolti, con l'analisi delle alternative scartate, la descrizione delle soluzioni e la reificazione dei design pattern adottati.

2.2.1 Sistema Polimorfico delle Valute e Motore di Conversione

1. Descrizione del problema

L'applicazione necessita di operare con classi di attività finanziarie eterogenee (valute a corso legale, criptovalute e titoli azionari). Tali attività presentano caratteristiche intrinseche profondamente disomogenee:

- Le valute fiat operano con convenzioni a 2 o 4 decimali e identificatori ISO standard (EUR, USD, GBP, ecc.).
- Le criptovalute presentano frazionabilità elevatissima (fino a 8–18 decimali) e richiedono identificatori specifici per le API di quotazione (es. `apiId` come `"bitcoin"`).
- I titoli azionari operano con ticker di borsa (es. `AAPL`, `GOOGL`) e mercati di riferimento regolamentati.

Inoltre, il calcolo della conversione tra due qualsiasi valute deve essere flessibile, estendibile e svincolato dall'implementazione specifica della sorgente dei tassi.

2. Soluzione proposta e alternative

- **Alternativa scartata (Classe monolitica o enum universale):** Raggruppare tutte le tipologie in un'unica enum con campi facoltativi avrebbe violato i principi *Open/Closed* (OCP) e *Single Responsibility* (SRP), obbligando a modificare codice esistente per aggiungere nuove valute o mercati.
- **Soluzione adottata:** Definizione dell'interfaccia comune `CurrencyUnit`, implementata in modo polimorfico da enum fortemente tipizzate: `FiatCurrency`, `CryptoCurrency` e `StockMarketCurrency`, oltre alla classe aperta `Currency` per asset personalizzati. Per l'esecuzione dei cambi è stata definita l'interfaccia `CurrencyConverter`, la cui implementazione principale `BasicCurrencyConverter` esegue calcoli rigorosi su `BigDecimal` con rounding bancario (`RoundingMode.HALF_UP`), incapsulando il risultato nel record immutabile `CurrencyConversionResult`.

3. Schema UML e Interazioni

La gerarchia delle valute e il motore di conversione sono strutturati come segue:

- `CurrencyUnit` (interfaccia radice): espone `getCode()`, `getSymbol()`, `getShortName()`, `getFullName()`, `getType()`.
- `FiatCurrency`, `CryptoCurrency`, `StockMarketCurrency`, `Currency`: implementano `CurrencyUnit`.
- `CurrencyConverter` (interfaccia): definisce `convert(amount, from, to)`.
- `BasicCurrencyConverter`: implementa `CurrencyConverter` delegando a `UniversalPriceService` per il recupero delle quotazioni relative al dollaro USD e calcolando il tasso cross applicato.

4. Pattern utilizzati e reificazione

- **Strategy Pattern:** L'interfaccia `CurrencyConverter` rappresenta la *Strategy* astratta per il calcolo delle conversioni. `BasicCurrencyConverter` costituisce la *Concrete Strategy* che esegue la triangolazione ponderata sui tassi correnti. Il client (`CurrencyConverterController` e `ConverterWidgetFX`) opera esclusivamente contro l'interfaccia, consentendo la sostituzione trasparente dell'algoritmo di conversione (es. per collaudo o con commissioni istituzionali).

- **Adapter Pattern e Composite Service:** I provider dedicati verso i servizi esterni (`CoinGeckoPriceProvider`, `AlphaPriceProvider` ed `ExchangeRateAPIClient`) fungono da *Adapter*, convertendo e normalizzando i formati JSON eterogenei dei diversi provider nei contratti uniformi del dominio (`PriceProvider` ed `ExchangeRateAPI`). Il componente `UniversalPriceService` opera come *Composite Service / Facade* orchestrando tali adapter in base alla tipologia di asset richiesta.

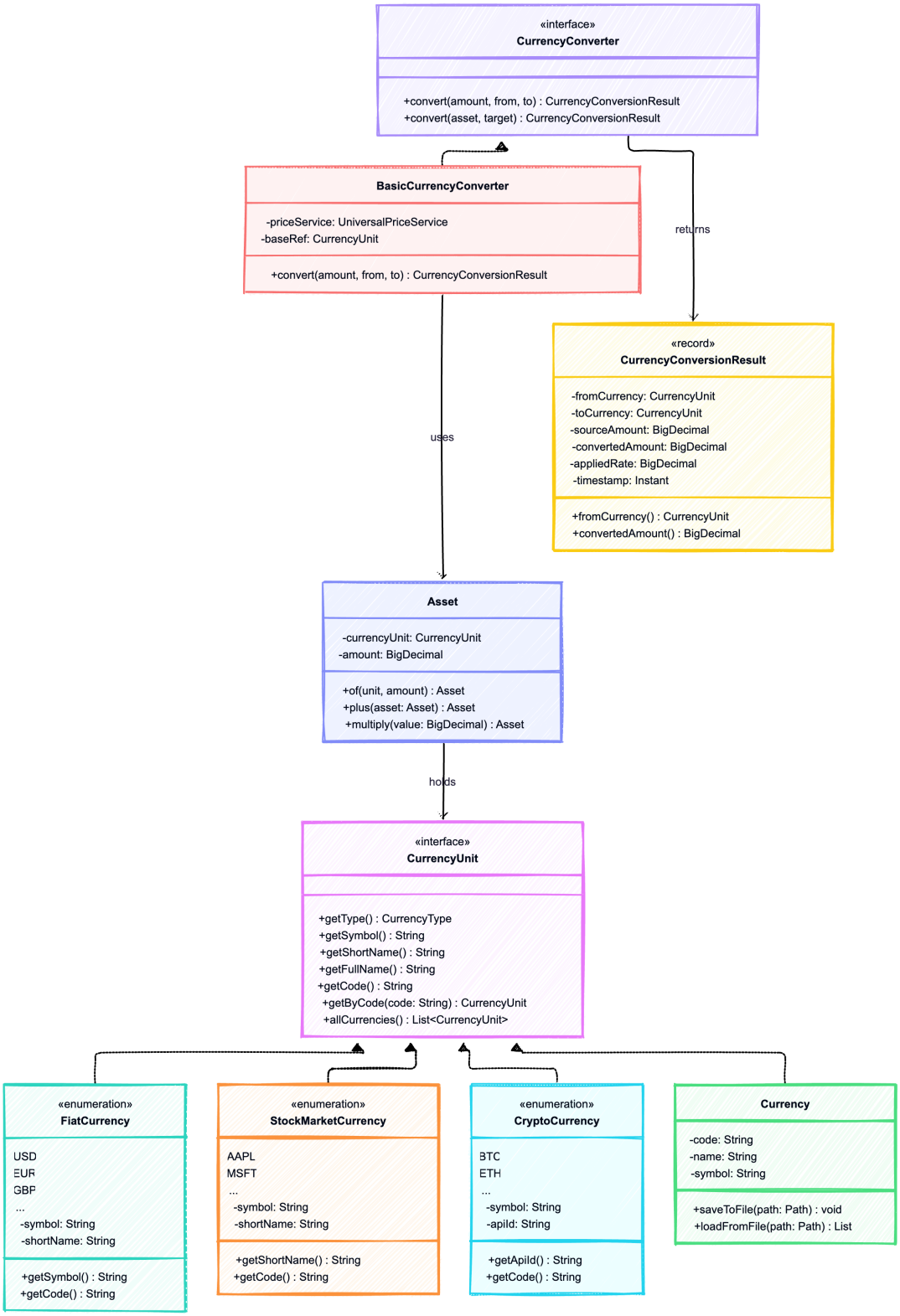

Figura 2.2: Il diagramma illusta gerarchia polimorfica di `CurrencyUnit`, `Asset` e motore di conversione.

2.2.2 Architettura dei Provider di Cambio con Caching Resiliente

1. Descrizione del problema

I servizi web esterni che forniscono le quotazioni di mercato delle diverse classi di attività (open.er-api.com per le valute legali fiat, CoinGecko v3 per le criptovalute, Alpha Vantage per i titoli azionari di borsa) impongono vincoli operativi e architetturali stringenti:

1. **Rate Limiting e Quote d'Uso Rigide:** I piani API gratuiti applicano soglie restrittive (es. 25–100 chiamate giornaliere su Alpha Vantage, 10–30 richieste al minuto su CoinGecko). Il superamento di tali limiti comporta il blocco temporaneo dell'indirizzo IP con codici di errore HTTP.

2. **Latenza e Rischio di Blocco della GUI:** Le chiamate HTTP su rete pubblica introducono tempi di risposta variabili (da centinaia di millisecondi a diversi secondi in caso di congestione). L'esecuzione sincrona non mediata sul thread applicativo JavaFX congelerebbe l'interfaccia.

3. **Resilienza Offline e Tolleranza ai Guasti:** L'applicazione deve garantire la piena continuità operativa anche in caso di assenza temporanea di connettività Internet, timeout di rete o manutenzione dei server remoti, senza perdere dati e senza generare crash inattesi.

2. Soluzione proposta e alternative

- **Alternativa scartata A (Chiamate dirette sincrone senza memorizzazione):** Interrogare direttamente l'endpoint REST ad ogni conversione o render grafico provocherebbe l'esaurimento delle quote API in pochi secondi di utilizzo e un'esperienza utente inaccettabile in caso di latenza.

- **Alternativa scartata B (Thread demone di polling periodico continuo):** Interrogare ciclicamente i server remoti in background consumerebbe inutilmente traffico dati e risorse computazionali anche quando l'utente non sta consultando il convertitore o il portafoglio.

- **Soluzione architetturale adottata:** Introduzione di un'architettura multilivello basata su:

    1. *Cache in memoria ad accesso immediato controllata da Time-To-Live (TTL):* Le risposte valide delle API vengono memorizzate associando un timestamp di aggiornamento (`Instant`). Ogni richiesta successiva pervenuta entro la finestra di validità viene servita in tempo costante ($O(1)$) dalla memoria, senza alcun traffico di rete;
    2. *Graceful Degradation e Provider Deterministico di Fallback:* In caso di errore HTTP, timeout o modalità offline, il sistema ricorre all'ultimo tasso valido memorizzato; qualora non vi sia alcuna quotazione pregressa (es. primo avvio offline o test automatizzati), interviene il componente `MockExchangeRateAPI`, che genera tassi realistici e riproducibili azzerando il rischio di interruzione del servizio.

3. Schema UML e Interazioni

L'interfaccia `ExchangeRateAPI` espone i metodi `getLatestRates(base)` e `getHistoricalRates(base, target, from, to)`. `ExchangeRateAPIImpl` implementa tale contratto mantenendo una mappa `cachedRates` associata a un timestamp `Instant lastUpdate`. Se la richiesta perviene entro la finestra di validità della cache, i dati vengono restituiti istantaneamente; in caso contrario, viene effettuata la chiamata HTTP aggiornando la cache solo a fronte di una risposta valida (HTTP 200). Parallelamente, `UniversalPriceService` coordina i provider dedicati per le diverse classi di asset, delegando a `PriceProvider` per quotazioni crittografiche e azionarie. L'architettura complessiva e le interazioni tra i provider sono illustrate nella Figura 2.3.

4. Pattern utilizzati e reificazione

- **Caching In-Memory con TTL e Fallback (Ispirato al Proxy):** `ExchangeRateAPIImpl` protegge l'accesso alla risorsa remota costosa (chiamate di rete su protocollo HTTP) incapsulando una cache in memoria controllata da un tempo di validità (*Time-To-Live*). Riprende il principio del Proxy nell'intercettare le richieste per servirle istantaneamente dalla memoria locale finché i dati sono validi, minimizzando le chiamate di rete.
- **Fallback Strategy e Mock Provider (Test Double deterministico):** `MockExchangeRateAPI` offre un'implementazione autonoma e affidabile del contratto `ExchangeRateAPI`, garantendo la continuità operativa in modalità offline e la determinazione nei test automatici unitari.

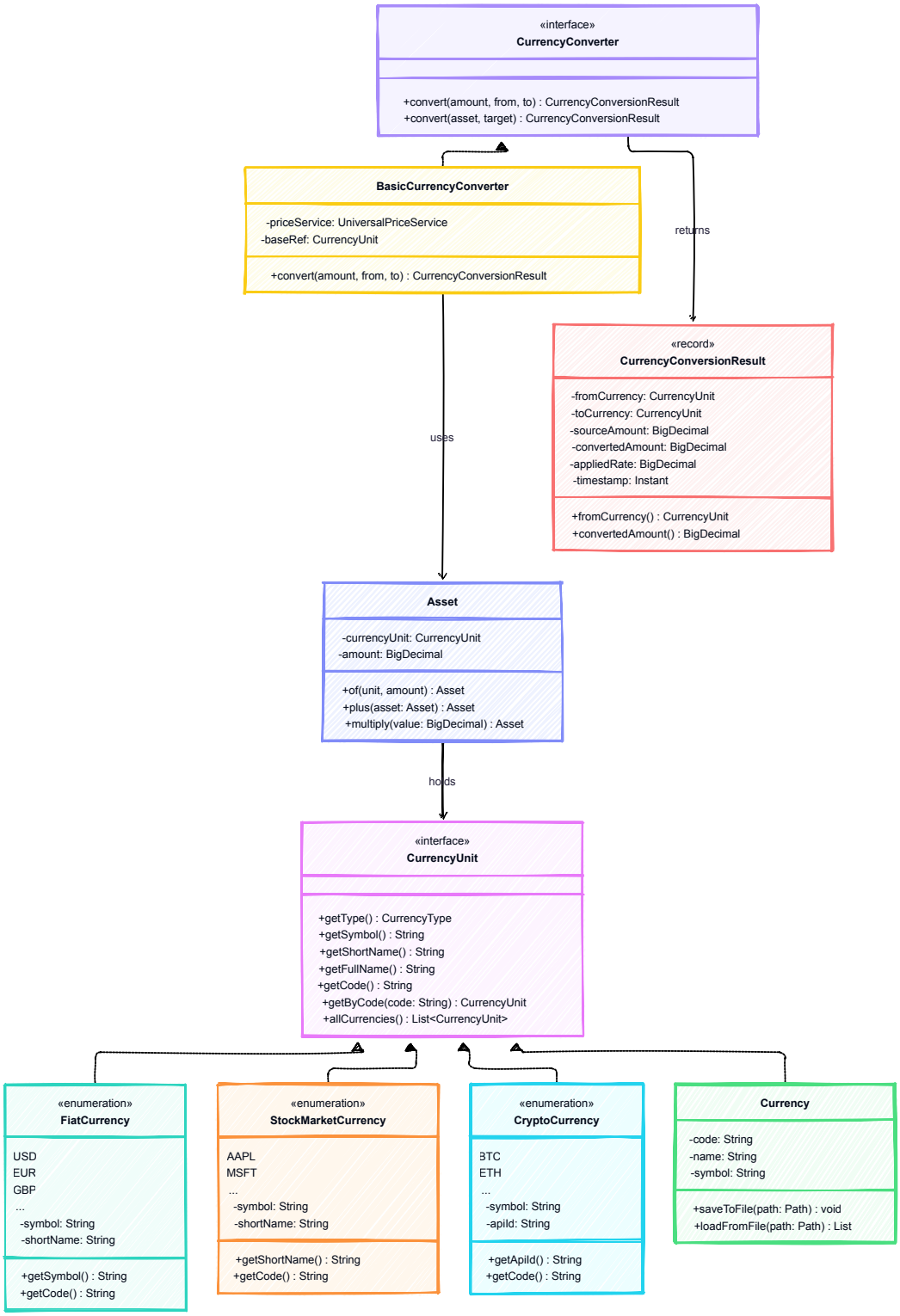

Figura 2.3: Diagramma delle classi: architettura dei provider di cambio, caching e fallback.

2.2.3 Monitoraggio dei Tassi, Allerte di Mercato e Watchlist Reattiva

1. Descrizione del problema

Un cruscotto finanziario completo richiede strumenti operativi di monitoraggio continuo:

1. **Tracciamento di Coppie Valutarie Dinamiche (*Watchlist*):** L'utente deve poter fissare un insieme personalizzato di coppie di scambio preferite per consultarne il tasso in tempo reale senza dover reimpostare ogni volta i menu di conversione;
2. **Allerte di Prezzo Reattive con Soglie Limite (*Alerts*):** L'utente deve poter definire soglie di cambio critiche (es. notificare se 1 EUR $>$ 1.10 USD oppure se 1 BTC $<$ 60000 USD), con direzione di attraversamento (*ABOVE* o *BELOW*);
3. **Preservazione del Disaccoppiamento MVC e Headless Testing:** La valutazione matematica delle condizioni di soglia non deve in alcun caso dipendere da primitive grafiche JavaFX (es. `javafx.scene.control.Alert`), preservando la rigorosa testabilità automatizzata.

2. Soluzione proposta e alternative

- **Alternativa scartata A (Controlli grafici inseriti direttamente nel Model):** Invocare finestre di dialogo modali all'interno della logica contabile violerebbe il principio di singola responsabilità (SRP), impedirebbe l'esecuzione dei test JUnit in ambienti headless e creerebbe dipendenze circolari tra Model e View.
- **Alternativa scartata B (Valutazione cablata nei controller delle singole form):** Duplicare il controllo delle soglie all'interno di ciascun widget visivo causerebbe ridondanza di codice e inconsistenze qualora la medesima coppia sia visualizzata sia nel convertitore sia nella watchlist.
- **Soluzione adottata:** Architettura a responsabilità rigorosamente segregate:
  - `CurrencyAlert`: record immutabile che incapsula la coppia valutaria, la soglia numerica e la direzione di allerta (`AlertDirection.ABOVE` o `BELOW`);
  - `CurrencyAlertService`: servizio puro del modello (privo di dipendenze da JavaFX) che custodisce l'elenco delle allerte attive ed espone il metodo funzionale `checkAlerts(rate, from, to)`, filtrando dichiarativamente tramite Stream le sole allerte che risultano effettivamente violate;
  - `MarketAlertView`: componente specializzato del layer View che riceve le allerte violate dal servizio e pianifica l'apertura del dialogo modale d'allerta sul thread grafico;
  - `WatchList`: componente di persistenza locale che mantiene un insieme unico (`Set<WatchlistPair>`) salvato atomicamente in formato JSON.

3. Schema UML e Interazioni

La separazione dei componenti per il monitoraggio reattivo dei mercati e per la gestione della watchlist è illustrata nel seguente diagramma:

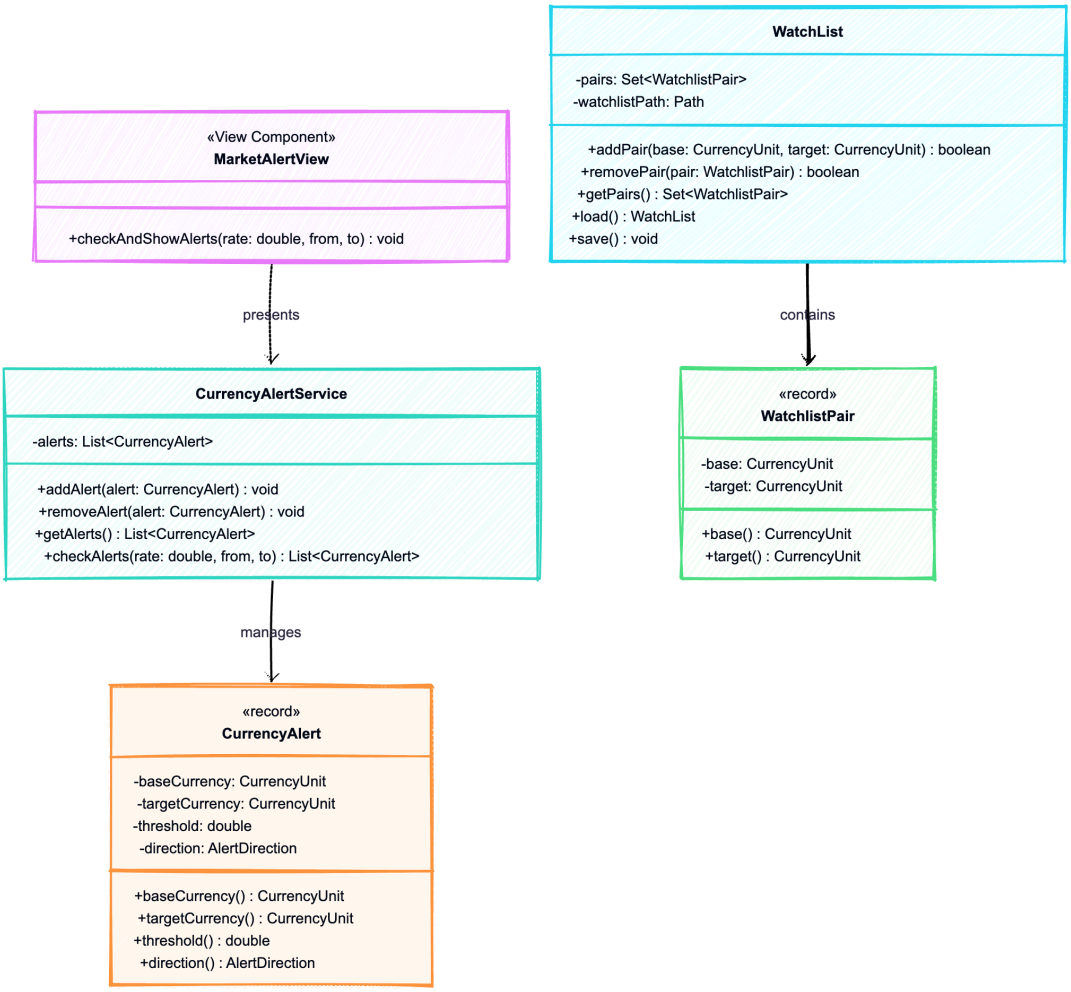

Figura 2.4: Diagramma delle classi: monitoraggio delle soglie, emissione delle allerte e gestione persistente della watchlist.

4. Pattern utilizzati e reificazione

- **Rule Engine Dichiarativo e Disaccoppiamento Reattivo:** `CurrencyAlertService` incapsula la logica pura di business per la valutazione delle soglie di allarme tramite Stream Java. La propagazione delle notifiche verso la vista (`MarketAlertView`) e verso gli altri moduli dell'applicazione è completamente disaccoppiata e demandata all'infrastruttura condivisa di Publish-Subscribe del `MessageBus`.
- **Defensive Design:** `WatchList` restituisce collezioni immutabili (`Collections.unmodifiableSet`) per proteggere lo stato interno da manipolazioni esterne non autorizzate.

2.2.4 Gestione delle Preferenze Utente, Temi Dinamici e Memento dello Stato

1. Descrizione del problema

L'ergonomia dell'applicazione prevede la completa personalizzazione dello spazio operativo: selezione della valuta di conto globale (con ricalcolo a catena dei totali di portafoglio), commutazione tra tema cromatico chiaro (*Light*) e scuro (*Dark*), regolazione della scala

tipografica e persistenza delle dimensioni geometriche della finestra principale. Questo scenario imponeva due requisiti architetturali:

1. **Ripristino dello Stato Pregresso (Undo/Restore):** Possibilità per l'utente di annullare modifiche accidentali in modifica del tema;
2. **Propagazione dei Fogli di Stile a Caldo (Hot Theme Swapping):** Aggiornamento istantaneo dell'aspetto visivo di tutte le scene e finestre modali attive senza richiedere il riavvio dell'applicazione e senza distruggere i nodi grafici dello SceneGraph.

2. Soluzione proposta e alternative

- **Alternativa scartata A (Variabili globali statiche sparse):** Mantenere impostazioni come variabili statiche distribuite nei controller genera accoppiamento occulto, impedisce il testing parallelo ed espone lo stato ad alterazioni concorrenti non controllate.
- **Alternativa scartata B (Riavvio forzato dell'applicazione):** Costringere l'utente a riavviare l'applicazione per applicare un tema o una valuta base comporta una grave degradazione dell'usabilità e la potenziale perdita di dati non salvati nelle form aperte.
- **Soluzione adottata:**
  - *Reificazione del Memento Pattern:* La classe `Settings` opera come *Originator*, incapsulando lo stato corrente e generando istantanee immutabili `SettingsSnapshot` (*Memento*) complete di timestamp, valuta e tema; il manager persistente `SettingsManager` e `SettingsController` fungono da *Caretaker*, archiviando lo storico delle istantanee e consentendo tramite `applyConfiguration(snapshot)` e `Settings.fromSnapshot(...)` di ripristinare uno stato passato in modo atomico;
  - *Registro centralizzato di stile nella View (`ThemeManager`):* Collocato nel package della vista (`it.unibo.unibodget.view.settings`), il manager centralizza la gestione dei fogli di stile. Il metodo `ThemeManager.applyThemeToScene(Scene)` manipola direttamente la collezione `scene.getStylesheets()`, iniettando a runtime le classi CSS radice (`.theme-dark` vs `.theme-light`) con ricalcolo immediato del rendering da parte del motore grafico JavaFX.

3. Schema UML e Interazioni

La struttura del sistema di gestione delle configurazioni, la reificazione del pattern Memento e la propagazione dello stile tramite Observer sono presentate nel seguente diagramma:

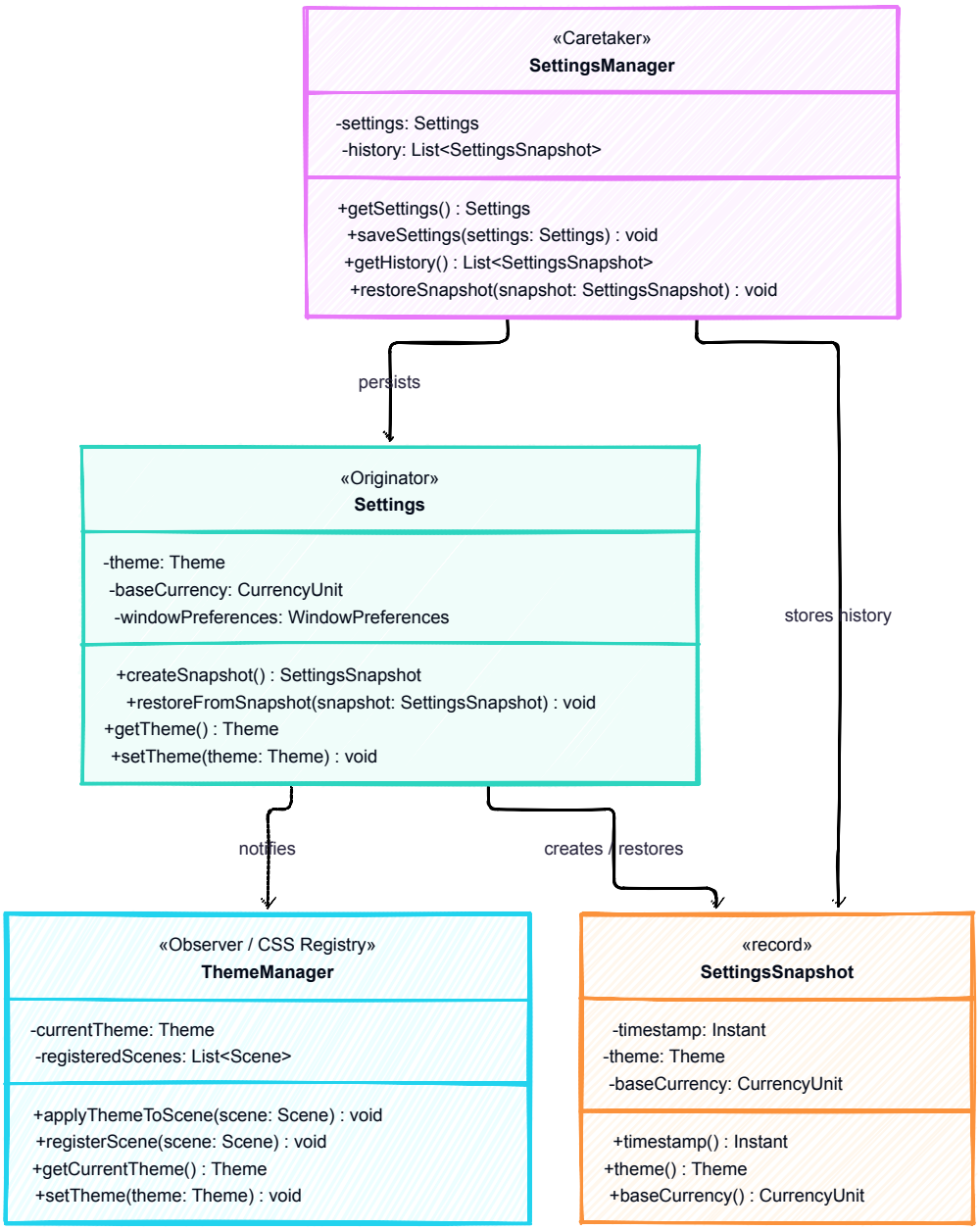

Figura 2.5: Diagramma delle classi: reificazione del Memento Pattern per le impostazioni e gestione dinamica del tema.

4. Pattern utilizzati e reificazione

- **Memento Pattern:** `Settings` riveste il ruolo di *Originator*; `SettingsSnapshot` costituisce il *Memento*; `SettingsManager` e `SettingsController` operano come *Caretaker*, gestendo la lista di snapshot senza violarne l'incapsulamento.
- **Observer Pattern:** `ThemeManager` notifica i listener registrati a ogni modifica di stile, sincronizzando istantaneamente la palette di tutti i componenti aperti.

2.2.5 Composizione, Accessibilità e Ottimizzazione della View

1. Descrizione del problema

La sezione di conversione non è un semplice modulo elementare, ma integra quattro widget visivi interattivi e complessi:

1. `ConverterWidgetFX`: calcolatore di cambio diretto tra valute con controlli di validazione e innesco delle allerte.
2. `BankConverterWidgetFX`: simulatore dei tassi bancari reali con spread e commissioni accessorie.
3. `CurrencyHistoryChartView`: grafico a linee che traccia l'andamento del cambio in un intervallo di date.
4. `WatchlistWidgetFX`: pannello di monitoraggio rapido delle coppie preferite.

Oltre alla necessità di aggregare questi componenti mantenendoli totalmente disaccoppiati dalla shell principale, si sono presentate due sfide ergonomiche e visive rilevanti:

- **Troncamento del selettore di direzione (*Above/Below*):** Nel pannello allerte del convertitore, il testo dell'etichetta veniva visualizzato tronco (es. "Abo...") a causa di un conflitto tra selettori CSS globali (`.theme-dark .combo-box`) e locali con padding eccessivo, unitamente a vincoli di layout troppo rigidi.
- **Adattamento dinamico del layout del pannello:** Quando la finestra viene ridimensionata e lo spazio orizzontale disponibile si riduce, l'interfaccia riorganizza automaticamente i componenti, disponendoli su più righe anziché mantenerli affiancati. In particolare, alcuni elementi vengono spostati nella parte inferiore del pannello per garantire una visualizzazione più ordinata e leggibile. Per gestire il contenuto che eccede lo spazio verticale disponibile, viene inoltre introdotta una barra di scorrimento sul lato destro, che consente di accedere a tutti gli elementi senza compromettere il layout.

2. Soluzione proposta e alternative

- **Alternativa scartata (Controller unico con codice FXML monolitico):** Riunire tutti i controlli in un unico mega-controller avrebbe generato un'anti-architettura difficilmente manutenibile e incline a violazioni di SpotBugs e Checkstyle.
- **Soluzione architetturale adottata:** Scomposizione in singoli widget autonomi dotati di visibilità package-private, orchestrati dalla classe `CurrencyConverterViewFX` ed esposti alla shell applicativa mediante `CurrencyConverterModule`.
- **Risoluzione dei conflitti di stile e specificità CSS:**
  - Per il selettore `directionBox`, è stata innalzata la specificità nel foglio di stile `theme.css` introducendo selettori contestualizzati (`.theme-dark .direction-box`, `.direction-box .cell`, `.direction-box .arrow-button`) con padding azzerato o controllato, stabilendo una larghezza minima di sicurezza (`125px`) e preferita (`130px`) e imponendo `HBox.setHgrow(directionBox, Priority.NEVER)` per prevenire compressioni indebite.

– **Adattamento dinamico del layout del pannello:** La disposizione dei widget è stata gestita mediante un componente `FlowPane`, configurato con una spaziatura orizzontale e verticale di `20px`, che consente il riposizionamento automatico degli elementi su righe successive quando lo spazio orizzontale disponibile non è sufficiente. L'intero contenuto della dashboard è stato inoltre racchiuso in un `ScrollPane`, configurato con `fitToWidth(true)` e barre di scorrimento impostate su `AS_NEEDED`, così da adattarsi alla larghezza della finestra e consentire lo scorrimento verticale quando il contenuto eccede l'area visibile.

3. Schema UML e Interazioni

L'architettura composita della vista del convertitore è strutturata come segue:

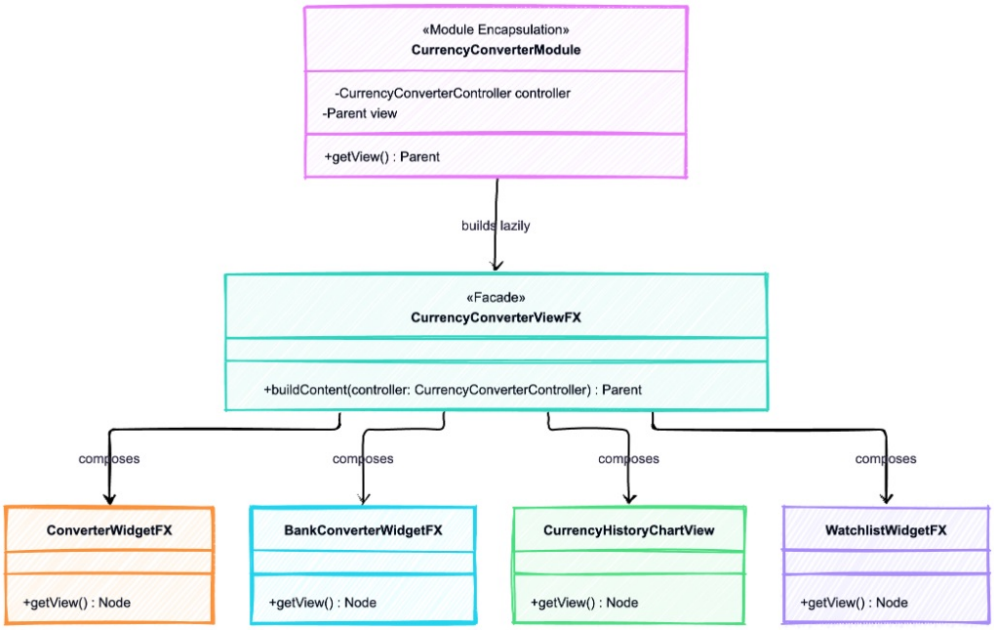

Figura 2.6: Diagramma delle classi: scomposizione modulare e composizione dei widget della vista del convertitore.

4. Pattern utilizzati e reificazione

- **Facade Pattern:** `CurrencyConverterViewFX` funge da *Facade* per l'intero sottosistema grafico del convertitore, nascondendo alla shell principale i dettagli dell'albero JavaFX e offrendo un punto d'ingresso pulito.
- **Incapsulamento Modulare e Lazy Initialization:** `CurrencyConverterModule` incapsula il ciclo di vita del modulo e adotta la *Lazy Initialization* per la costruzione dell'albero grafico JavaFX (`CurrencyConverterViewFX`), istanziando i nodi visivi solo al primo effettivo accesso e ottimizzando l'impronta di memoria e i tempi di bootstrap.

2.2.6 Risoluzione Resiliente del File System e Autoconsistenza dei Dati (AppDataPathResolver)

1. Descrizione del problema

Nelle prime versioni dell'applicazione, ciascuna classe responsabile della persistenza sul disco (tra cui `SettingsManager`, `ThemeList`, `InvestmentAccountManager`, `CashAccountManager`, `CategoryManager`, `BankList`, `Currency` e `WatchList`) definiva in modo rigido e duplicato il percorso fisico dei file (`Path.of("data/json/...")`). Tale approccio presentava gravi limiti di progettazione e usabilità:

1. **Violazione del Single Source of Truth:** La moltiplicazione di costanti di percorso cablate impediva qualsiasi riconfigurazione trasparente della cartella dati, richiedendo modifiche ad otto classi distinte in caso di variazione strutturale.
2. **Fragilità d'avvio da sottocartelle o ambienti eterogenei:** Se l'applicazione veniva eseguita impostando come working directory una cartella diversa dalla radice del repository (ad esempio lanciando test o task da una sottodirectory o da contesti IDE esterni), l'accesso relativo a `./data` falliva, provocando il mancato caricamento dei dati utente.
3. **Dispersione accidentale dei dati:** In caso di riposizionamento involontario della cartella dati sul computer dell'utente (ad esempio spostamento su Scrivania o Download), l'applicazione non disponeva di alcun meccanismo euristico di rilevamento e procedeva alla generazione forzata di dati vuoti, con potenziale disorientamento per l'utente.

2. Soluzione proposta e alternative

- **Alternativa scartata (Passaggio obbligatorio di parametri da riga di comando):** Richiedere all'utente di specificare sempre un parametro JVM (es. `-Dunibodget.data.dir=...`) avrebbe peggiorato l'usabilità immediata del software.
- **Alternativa scartata (Scansione globale ricorsiva del disco):** Una ricerca cieca sull'intero albero del file system avrebbe introdotto ritardi inaccettabili in fase di avvio, esponendosi al rischio di blocchi dovuti a permessi di accesso o collegamenti ciclici.
- **Soluzione adottata:** Centralizzazione della risoluzione in una classe dedicata, `AppDataPathResolver`, fondata su una gerarchia rigorosa a quattro livelli di priorità:
  1. **Passo 1 (Data locale standard):** Se la directory `./data` esiste nella cartella di lavoro corrente, viene selezionata ed impiegata immediatamente. Questa scelta azzera qualsiasi latenza di scansione (*zero search overhead*) e assicura il 100% di continuità e stabilità rispetto al comportamento standard.
  2. **Passo 2 (Ricerca euristica intelligente delimitata):** Qualora la cartella locale non sia presente, il resolver innesca un'indagine controllata e protetta da vincoli stringenti:
     - Esamina percorsi adiacenti frequenti (`../data`, `../../data`, `app/data`, `~/.unibodget/data`).
     - Scansiona le directory utente canoniche (Desktop, Documents, Downloads e radice superiore) con profondità massima limitata a 3 livelli (`MAX_SEARCH_DEPTH`

= 3), fermando l'ispezione al raggiungimento di 500 cartelle visitate e saltando a priori rami nascosti (`.git`, `.gradle`) e cartelle di build (`build`, `bin`, `target`, `node_modules`, `Library`).
  - Sottopone ciascun candidato a **validazione semantica**: il file `json/settings/Settings.jso` deve esistere, essere leggibile e contenere campi JSON proprietari di UniBodget (`baseCurrency`, `theme`), eliminando alla radice il rischio di falsi positivi su cartelle omonime di terze parti.
3. **Passo 3 (Parametro esplicito opzionale):** Se la ricerca euristica non rileva cartelle preesistenti, il resolver verifica la presenza della proprietà JVM di sistema `unibodget.data.dir` o della variabile d'ambiente `UNIBODGET_DATA_DIR`.
4. **Passo 4 (Clean fallback con self-healing):** Come garanzia conclusiva di funzionamento, il resolver ricade sul percorso standard `Path.of("data")`. Qualora i file non esistano, `ModelFileManager` interviene in modalità di *self-healing*, estraendo automaticamente i template JSON predefiniti inclusi nel classpath applicativo (`resources/json/`).

3. Schema UML e Interazioni

Il ruolo centrale di `AppDataPathResolver` come punto di disaccoppiamento e l'interazione con le classi persistenti sono illustrati nel seguente diagramma:

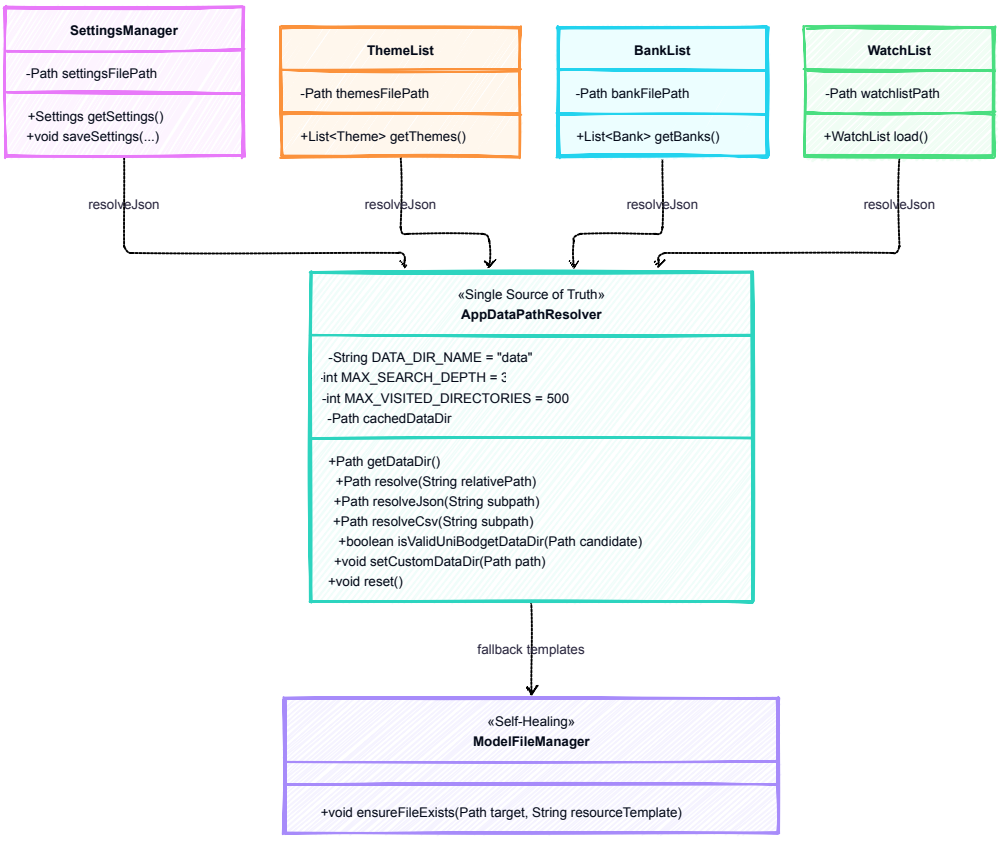

Figura 2.7: Diagramma delle classi: architettura del percorso dati centralizzato e relazioni con i manager di persistenza.

4. Pattern utilizzati e reificazione

- **Risoluzione Euristica a Priorità a Cascata:** La selezione della directory dati incapsula una sequenza di strategie di lookup ordinate per priorità (directory personalizzata da test, root di repository di sviluppo o percorso standard utente), arrestandosi al verificarsi della prima condizione valida.
- **Single Source of Truth:** Tutte le 8 classi persistenti sono state disaccoppiate e collegate ad `AppDataPathResolver.resolveJson(...)` e `resolveCsv(...)`, garantendo che qualsiasi modifica logistica della cartella dati si propaghi istantaneamente e uniformemente a tutta l'applicazione.
- **Design for Testability:** La classe mette a disposizione i metodi `setCustomDataDir(Path)` e `reset()`, consentendo alle suite di test automatizzati di operare su directory temporanee isolate (`@TempDir`) senza interferire con lo stato persistente reale dell'applicazione.

2.2.7 Normalizzazione del Parsing e Validazione dell'Input Monetario (MonetaryInputParser)

1. Descrizione del problema

Nelle applicazioni gestionali e finanziarie, l'interazione con l'utente sui campi di inserimento numerico (importi di spesa, quote di budget, tassi di cambio, soglie di allerta e commissioni) presenta una forte eterogeneità di convenzioni sintattiche:

- La notazione standard internazionale impiega il punto (`.`) come delimitatore decimale (es. `1250.50`).
- La convenzione italiana ed europea fa uso della virgola (`,`) come separatore dei decimali (es. `1250,50`).
- L'utente inserisce frequentemente spaziature non intenzionali prima, dopo o all'interno dell'importo (es. copia-incolla con spazi di raggruppamento delle migliaia come `"1 250,50"` o apostrofi di formattazione bancaria come `"1'250.50"`).

Nel design originario, ciascuna finestra di dialogo e ciascun widget del modulo (`ConverterWidgetFX`, `BankConverterWidgetFX`, `CurrencyHistoryChartView`, unitamente ai dialoghi della dashboard `NewTransactionDialog` ed `EditBudgetDialog`) implementava logiche di parsing frammentate e non uniformi: alcune chiamavano direttamente `new BigDecimal(text)`, che solleva improvvisamente un'eccezione non controllata `NumberFormatException` in presenza di virgole o caratteri non canonici; altre applicavano sostituzioni parziali (`.replace(',', '.')`) senza alcuna validazione formale della struttura; altre ricorrevano a `Double.parseDouble(text)`, esponendosi alle tipiche imperfezioni di arrotondamento binario a virgola mobile.

2. Soluzione proposta e alternative

- **Alternativa scartata (Gestione isolata con blocchi try-catch in ciascun controller):** Catturare le eccezioni localmente nelle singole classi di vista avrebbe moltiplicato il codice duplicato e lasciato inalterata la fragilità sintattica delle form.
- **Alternativa scartata (Blocco preventivo con regex restrittive nei campi di testo):** Impedire a priori l'inserimento della virgola tramite listener sui `TextField` avrebbe peggiorato l'usabilità per gli utenti con tastierino numerico o localizzazione italiana.

- **Soluzione adottata:** Progettazione della classe di utilità dedicata `MonetaryInputParser` all'interno del package `model.currency`, concepita come *Single Source of Truth* per l'analisi e la validazione degli input monetari dell'intera applicazione:

  1. **Sanitizzazione robusta (`sanitize`):** Rimuove spazi vuoti iniziali e finali, strip degli spazi di raggruppamento interni, rimozione di apostrofi ed eventuale normalizzazione della virgola nel punto decimale standard.
  2. **Parsing non bloccante con `Optional<BigDecimal>` (`parse`):** Invece di propagare eccezioni a runtime, il metodo restituisce un `Optional` vuoto in caso di input non parsabile, nullo o non conforme alla grammatica numerica rigorosa (`^-?\\d+(\\.\\d+)?$`), favorendo una programmazione funzionale ed elegante nei chiamanti (`.orElse(null)` o `.ifPresent()`).
  3. **Validazioni di dominio con vincoli algebrici:** Fornisce metodi specializzati ad alto livello:
     - `parsePositive(String)`: accetta esclusivamente importi strettamente positivi (> 0), ideale per somme da convertire o transazioni;
     - `parseNonNegative(String)`: ammette anche il valore zero (≥ 0), indispensabile per commissioni bancarie o spread facoltativi;
     - `parsePositiveWithMaxScale(String, int)`: valida e limita il numero massimo di cifre decimali ammesse in relazione alla granularità della valuta (es. 2 decimali per FIAT, scale maggiori per Crypto).
  4. **Adapter per primitive numeriche (`parseDouble`, `parsePositiveDouble`, `parseNonNegativeDouble`):** Offre un'interfaccia coerente anche per i componenti che richiedono coordinate numeriche per grafici o soglie temporanee, garantendo la protezione da `NaN` e infiniti.

3. Schema UML e Interazioni

L'architettura del parser monetario centralizzato e i componenti di vista che ne fanno uso sono riassunti nel diagramma delle classi seguente:

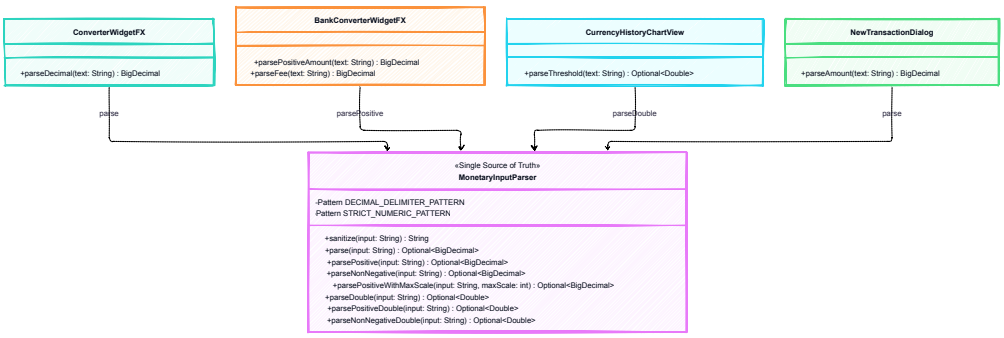

Figura 2.8: Diagramma delle classi: architettura di validazione e parsing dell'input monetario e relative interazioni con le viste.

4. Pattern utilizzati e reificazione

- **Utility Class Pattern:** Costruttore privato difensivo che impedisce l'istanziazione anche via reflection, con metodi statici puri, deterministici e privi di effetti collaterali (*side-effect free*).

- **Optional Pattern & Eliminazione delle Eccezioni come Flusso di Controllo:** Sostituzione dell'anti-pattern del controllo di flusso basato su eccezioni (*Exception-as-Control-Flow*) con costrutti funzionali espliciti e type-safe.
- **DRY (Don't Repeat Yourself):** Tutte le viste dell'applicazione (convertitore ordinario, convertitore bancario, grafico storico, dialoghi di transazione e di budget) sono state uniformate facendole convergere sull'uso esclusivo di `MonetaryInputParser`.

2.3 Design Dettagliato (Favour Osaretin Emuwahen)

In questa sezione vengono documentate le decisioni architetturali, i pattern di progettazione e le motivazioni tecniche adottate nello sviluppo del sottosistema di mia competenza: la Dashboard interattiva, la contabilità dei Portafogli di Cassa (*Cash Wallets*), la costruzione e validazione delle Transazioni Contanti (*Cash Transactions*), l'analisi comparativa dei Trend Mensili (*Monthly Insights*), il monitoraggio dinamico delle soglie di Budget e il Filtraggio declarativo dello storico movimenti.

2.3.1 Coordinamento e Disaccoppiamento della Dashboard (Facade e Passive View)

1. Descrizione del problema

La Dashboard rappresenta il centro operativo dell'applicazione: deve aggregare ed esporre simultaneamente una pluralità di informazioni eterogenee:

- Saldo totale e controvalore convertito nella valuta di riferimento globale;
- Ripartizione percentuale e scomposizione delle uscite per categoria;
- Stato di avanzamento del budget mensile con percentuale di consumo;
- Posizione debitoria/creditoria netta verso amici (*Friend Loans*);
- Indicatori di tendenza mese su mese (*Monthly Insights*);
- Cronologia dei movimenti recenti con filtri dinamici di ricerca.

Consentire al controller o alla vista di interrogare direttamente i singoli servizi di dominio (gestione conti, categorie, calcolo budget, prestiti, statistiche) avrebbe introdotto un accoppiamento elevatissimo (*tight coupling*), violando la Legge di Demetra e rendendo la View dipendente dai dettagli interni del Model. Inoltre, esporre entità mutabili del dominio alla GUI avrebbe compromesso l'integrità contabile.

2. Soluzione proposta e alternative

- **Alternativa scartata (Controller multi-servizio accoppiato):** Far dialogare il controller direttamente con ciascun servizio avrebbe portato alla creazione di una *God Class* con decine di dipendenze dirette, rendendo complessa la manutenzione e impossibili i test in isolamento.
- **Alternativa scartata (Passaggio di entità di dominio alla View):** Far consumare direttamente `CashAccount` e `Category` ai componenti JavaFX espone la rappresentazione interna a manipolazioni esterne non controllate (*representation exposure*).

- **Soluzione adottata:** Introduzione del pattern Facade tramite l'interfaccia `DashboardFacade` e la classe `DefaultDashboardFacade`. Il controller invoca un unico metodo ad alto livello (`loadDashboard()`), che orchestra i sotto-servizi e restituisce un oggetto istantanea immutabile: `DashboardSnapshot`. Il controller trasforma quindi tale snapshot in una gerarchia di record immutabili di presentazione (`DashboardViewState`), consegnati alla vista passiva (`DefaultDashboardView`) per il solo rendering grafico.

3. Schema UML e Interazioni

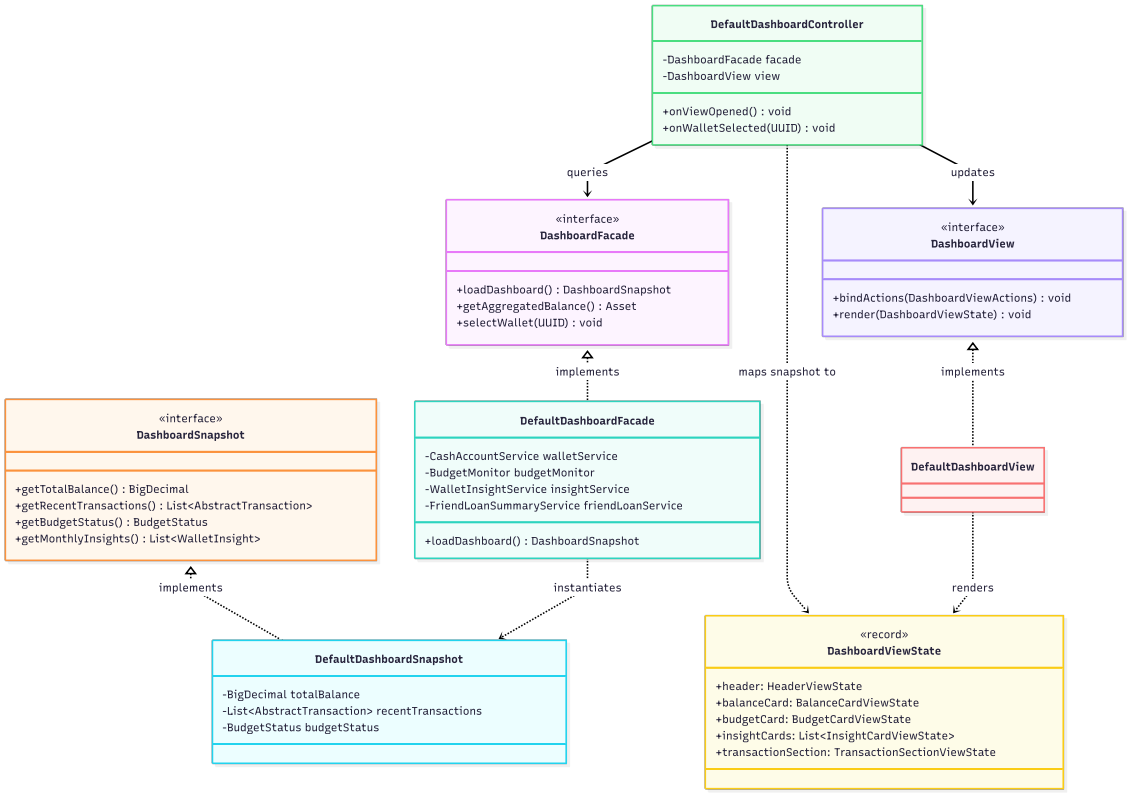

Figura 2.9: Reificazione del Facade Pattern e della Passive View per il sottosistema Dashboard.

4. Pattern utilizzati e reificazione

- **Facade Pattern:** `DefaultDashboardFacade` agisce come facciata unificata, incapsulando `CashAccountService`, `CategoryService`, `BudgetMonitor`, `FriendLoanSummaryService` e `WalletInsightService`.
- **Value Object / Snapshot Pattern:** `DashboardSnapshot` raccoglie i dati aggregati del modello all'istante di campionamento, garantendo consistenza atomica.
- **Passive View (MVC):** `DefaultDashboardView` non accede mai al modello; riceve lo stato `DashboardViewState` dal controller e notifica le interazioni utente tramite l'interfaccia `DashboardViewActions`.

- **Observer Pattern:** `DefaultDashboardController` implementa `WalletObserver`; quando il servizio portafogli notifica un cambiamento, il controller interroga la facciata e aggiorna reattivamente la vista.

2.3.2 Costruzione Fluente e Validazione delle Transazioni (Builder e Factory Pattern)

1. Descrizione del problema

Nel contesto della contabilità di cassa, una transazione monetaria (`CashTransaction`) presenta una marcata eterogeneità di parametri:

- Attributi obbligatori: identificativo univoco (`UUID`), asset monetario con importo e valuta (`Asset`), categoria contabile (`Category`), data di registrazione (`LocalDate`) e causale;
- Attributi contestuali/opzionali: note aggiuntive descrittive, nominativo dell'amico (`friendName`) e identificativo del prestito correlato (`friendLoanId`) nel caso di prestiti fiduciari.

L'approccio tradizionale basato su costruttori sovraccaricati (*telescoping constructors*) generava confusione e rischio di inversione di argomenti del medesimo tipo (stringhe). D'altro canto, consentire l'uso di setter avrebbe reso l'oggetto mutabile, violando l'integrità contabile. Era inoltre necessario automatizzare la corretta assegnazione del segno algebrico degli importi (es. importi negativi per spese e prestiti erogati, positivi per entrate e restituzioni).

2. Soluzione proposta e alternative

- **Alternativa scartata (Costruttori multipli telescopici):** Impraticabile con l'aumentare dei campi; obbliga a passare parametri null per i valori facoltativi.
- **Alternativa scartata (JavaBeans mutabili):** Scartata categoricamente; in un'applicazione contabile, permettere la modifica di data, importo o categoria post-costruzione espone a corruzioni dello storico.
- **Soluzione adottata:** Implementazione del pattern Builder come classe interna statica fluente (`CashTransaction.Builder`), coadiuvata dalla classe specializzata Factory (`CashTransactionFactory`). Il builder valida rigorosamente le invarianti nel metodo terminale `build()` (es. vincolo di presenza obbligatoria di `friendName` se la transazione rappresenta un prestito), garantendo che nessuna transazione possa esistere in uno stato incoerente.

3. Schema UML e Interazioni

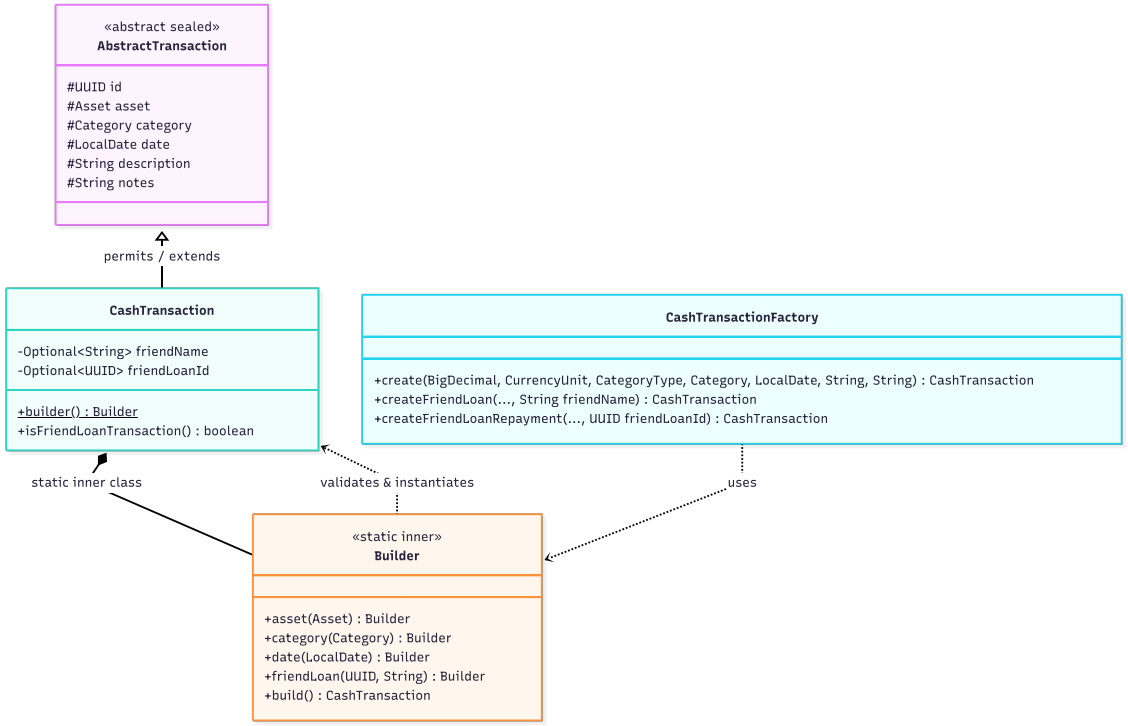

Figura 2.10: Reificazione dei pattern Builder e Factory per la creazione sicura di CashTransaction.

4. Pattern utilizzati e reificazione

- **Builder Pattern:** `CashTransaction.Builder` consente la composizione fluente passo-passo dei parametri, validando le invarianti di business prima di restituire l'oggetto immutabile.
- **Factory Pattern:** `CashTransactionFactory` incapsula la costruzione validata delle transazioni (`create`, `createFriendLoan`, `createFriendLoanRepayment`), normalizzando automaticamente la polarità algebrica degli importi (`Asset.amount().negate()` per uscite e prestiti erogati, positivo per entrate e rimborsi) e garantendo la coerenza tra `CategoryType` e i metadati fiduciari (`friendName`, `friendLoanId`).

2.3.3 Monitoraggio Dinamico e Strategie di Allerta del Budget (Strategy Pattern)

1. Descrizione del problema

Il controllo delle spese rispetto al budget mensile richiede di valutare la spesa cumulativa corrente rispetto al tetto limite prefissato dall'utente e a una soglia percentuale di avviso (es. 80%). In base al consumo calcolato, il sistema deve emettere un responso qualitativo differenziato (`SAFE`, `WARNING`, `CRITICAL`), corredato da messaggi contestuali e badge cromatici di allerta nella dashboard. La logica di valutazione doveva essere totalmente estendibile, consentendo l'introduzione futura di nuovi criteri di allarme (es. proiezioni

di spesa predittiva a metà mese o budget per singola categoria) senza dover modificare il monitor centrale.

2. Soluzione proposta e alternative

- **Alternativa scartata (Istruzioni condizionali annidate nel monitor):** L'uso di ramificazioni if-else o switch cablate all'interno di `DefaultBudgetMonitor` avrebbe violato il principio Open/Closed (OCP), imponendo modifiche al codice collaudato ad ogni variazione o aggiunta di criteri di soglia.
- **Soluzione adottata:** Adozione dello Strategy Pattern. L'interfaccia `BudgetAlertStrategy` definisce il contratto formale per la valutazione algoritmica della spesa tramite i metodi `applies(...)` ed `evaluate(...)`. Il monitor (`DefaultBudgetMonitor`) delega la valutazione sequenziale a una catena ordinata di strategie concrete (`CriticalBudgetStrategy`, `WarningBudgetStrategy`, `SafeBudgetStrategy`), ottenendo come responso lo stato contabile immutabile espresso dall'enumerazione `BudgetStatus` (`SAFE`, `WARNING`, `CRITICAL`).

3. Schema UML e Interazioni

L'architettura dello Strategy Pattern e le relative classi concrete di allerta sono illustrate nella Figura 2.11.

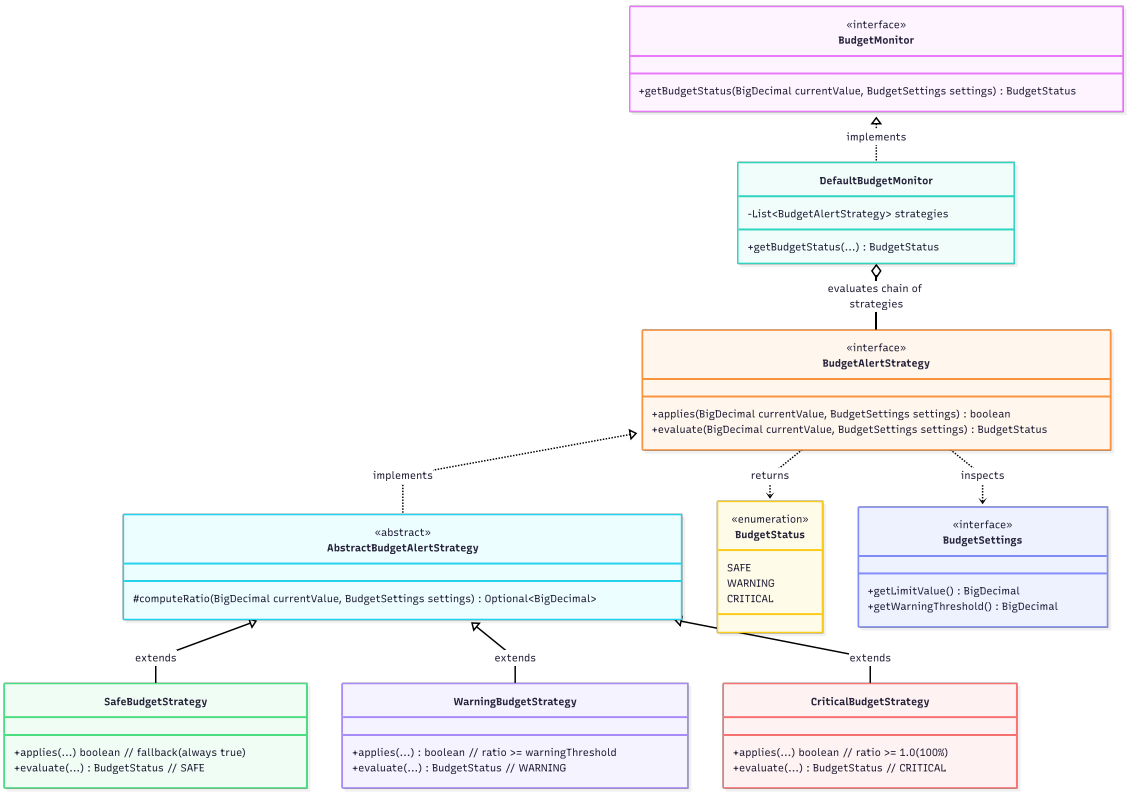

Figura 2.11: Reificazione dello Strategy Pattern per la valutazione delle soglie di allarme del Budget.

4. Pattern utilizzati e reificazione

- **Strategy Pattern:** `BudgetAlertStrategy` costituisce la Strategy astratta; `SafeBudgetStrategy`, `WarningBudgetStrategy` e `CriticalBudgetStrategy` rappresentano le Concrete Strategies. Il client (`DefaultBudgetMonitor`) agisce da Context, mantenendo completa indipendenza dalle specifiche condizioni di soglia.
- **Open/Closed Principle (OCP):** È possibile introdurre nuove strategie di allerta creando nuove classi che implementano l'interfaccia, senza alterare il codice esistente del monitor.

2.3.4 Algoritmo di Analisi Comparativa Mensile (Template Method Pattern)

1. Descrizione del problema

La dashboard include schede sintetiche per l'analisi comparativa mese-su-mese (*Monthly Insights*) relative a tre indicatori finanziari fondamentali: *1) Spending* (uscite), *2) Income* (entrate), *3) Net Savings* (risparmio netto). Ciascuno di questi calcoli condivide la medesima struttura algoritmica sequenziale:

1. Partizionare lo storico transazioni isolando il mese corrente e il mese solare antecedente;
2. Aggregare la metrica monetaria di interesse per ciascuno dei due periodi;
3. Calcolare la variazione percentuale (*delta percentage*), gestendo i casi limite di divisione per zero quando il mese precedente ha valore nullo;
4. Determinare la qualità semantica del trend (`POSITIVE`, `NEGATIVE`, `NEUTRAL`);
5. Istanziare il record `WalletInsight` con testo formattato e badge di tendenza.

Duplicare questa sequenza logica in tre servizi distinti avrebbe generato ridondanza di codice e rischi di calcolo difforme.

2. Soluzione proposta e alternative

- **Alternativa scartata (Metodo monolitico procedurale):** Raggruppare i tre calcoli in un unico blocco condizionale avrebbe creato un metodo voluminoso, fragile e difficilmente manutenibile.
- **Soluzione adottata:** Adozione del pattern Template Method. La classe astratta `AbstractMonthlyInsightTemplate` fissa lo scheletro invariante dell'algoritmo nel metodo template finale `generate(current, previous)`, standardizzando il calcolo del delta percentuale, gli arrotondamenti con scala esplicita e la gestione dei casi limite con baseline nulla (`handleZeroBaseline`). Essa delega alle sottoclassi concrete i metodi primitivi hook per il titolo (`getTitle`), l'attribuzione semantica della direzione della variazione (`getIncreaseTrend`, `getDecreaseTrend`) e la formattazione descrittiva dei messaggi di sintesi (`formatIncrease`, `formatDecrease`, `formatUnchanged`).

3. Schema UML e Interazioni

La gerarchia delle classi e il flusso algoritmico del Template Method sono schematizzati nella Figura 2.12.

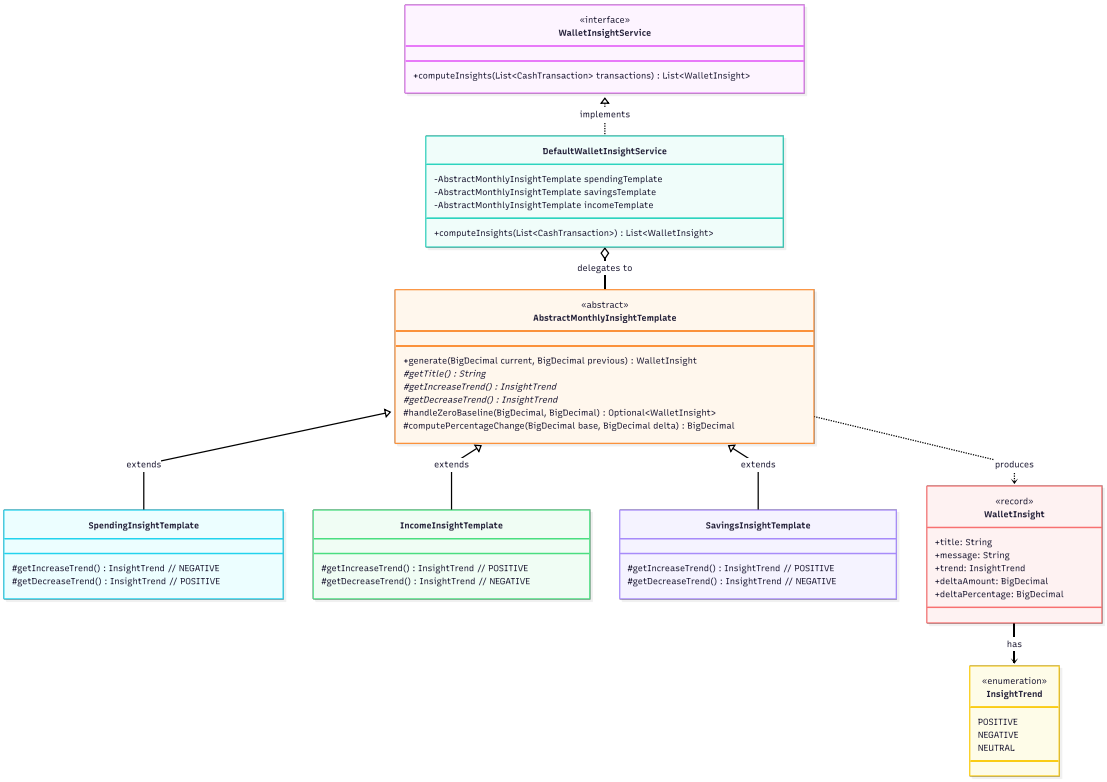

Figura 2.12: Reificazione del Template Method Pattern per il calcolo comparativo dei Monthly Insights.

4. Pattern utilizzati e reificazione

- **Template Method Pattern:** Il metodo `generate()` di `AbstractMonthlyInsightTemplate` definisce l'ordine di esecuzione immutabile (filtraggio → calcolo metriche → delta percentuale con gestione divisione per zero → determinazione trend).
- **Inversione del Controllo (Hollywood Principle):** Le classi concrete (`SpendingInsightTemplate`, `IncomeInsightTemplate`, `SavingsInsightTemplate`) non controllano il flusso; implementano solo la specifica metrica contabile e la semantica del trend (ad esempio, per le uscite un delta di spesa negativo è un trend `POSITIVE`, mentre per le entrate è considerato `NEGATIVE`).

2.3.5 Filtraggio Composito ed Estendibile della Cronologia (Specification Pattern)

1. Descrizione del problema

La tabella dei movimenti finanziari della Dashboard offre all'utente una barra di ricerca avanzata con filtri combinabili: intervallo temporale (*from/to date*), tipologia contabile (*Tutte, Entrate, Spese, Prestiti*), categoria, ricerca testuale su causale e note, e ordinamento dinamico. Una gestione procedurale a controlli condizionali concatenati avrebbe innescato un'esplosione combinatoria ($2^4 = 16$ rami logici), rendendo il codice fragile e prono a regressioni; l'introduzione di un nuovo filtro avrebbe inoltre richiesto modifiche invasive all'intero servizio.

2. Soluzione proposta e alternative

- **Alternativa scartata (Filtraggio procedurale con if annidati):** Scartata per la natura non componibile e l'elevata fragilità del flusso imperativo.
- **Alternativa scartata (Predicati generici anonimi):** L'uso di predicati standard (`Predicate<CashTransaction>`) dispersi nel controller avrebbe privato il dominio di astrazioni formali nominative e riusabili.
- **Soluzione adottata:** Adozione dello Specification Pattern (DDD). L'interfaccia formale `TransactionSpecification` estende `Predicate<CashTransaction>` fornendo metodi algebrici di default (`and`, `or`, `not`). Le specifiche atomiche (`CategoryNameSpecification`, `CategoryTypeSpecification`, `DateRangeSpecification`, `KeywordSpecification`) vengono assemblate dalla factory `TransactionSpecifications.fromCriteria(criteria)`. Il servizio `DefaultTransactionHistoryFilterService` riceve i criteri, recupera la specifica composta e filtra lo stream in un singolo passaggio dichiarativo.

3. Schema UML e Interazioni

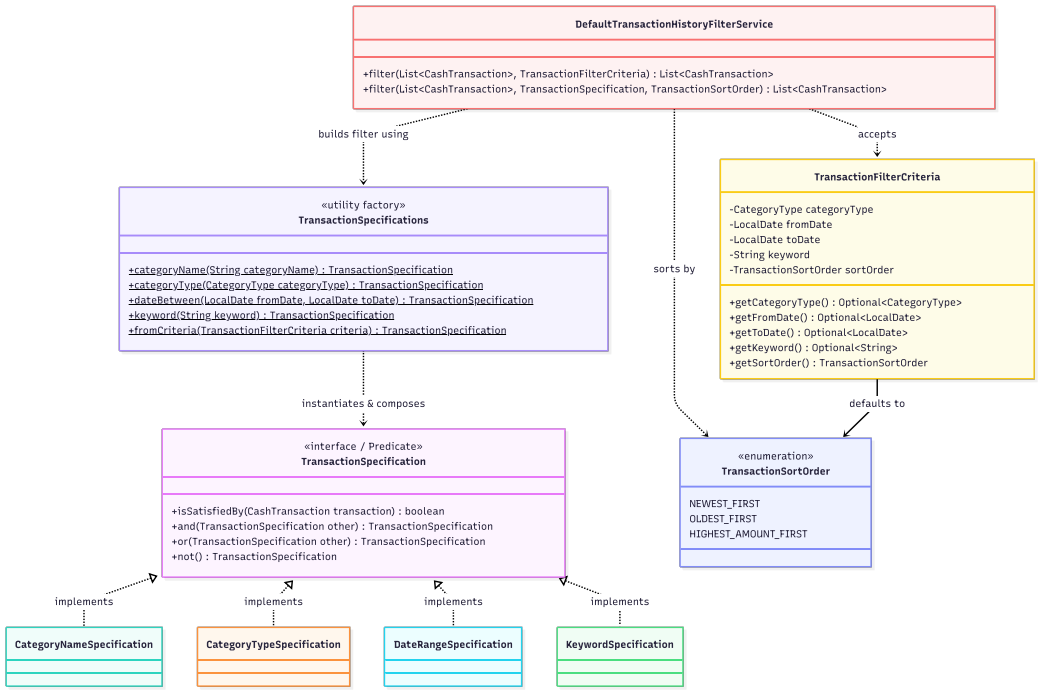

Figura 2.13: Reificazione dello Specification Pattern per il filtraggio avanzato e componibile delle transazioni.

4. Pattern utilizzati e reificazione

- **Specification Pattern:** Separa la logica del predicato contabile dall'iterazione sulla collezione; ogni specifica rispetta il principio di singola responsabilità (SRP) ed è collaudabile in completo isolamento.
- **Factory Method / Combiner:** La classe `TransactionSpecifications` centralizza l'istanziazione e la composizione logica (`spec.and(...)`) a partire dal record `TransactionFilterCriteria`.

- **Functional Pipeline e Defensive Copying:** Il filtraggio sfrutta `Stream.filter` con il predicato `isSatisfiedBy` e restituisce collezioni immutabili (`List.copyOf`), proteggendo l'integrità contabile da manipolazioni esterne.

2.4 Design Dettagliato (Daniele Zappalà)

In questa sezione vengono presentati i quattro problemi di progettazione principali affrontati e risolti da **Daniele Zappalà**, riguardanti la gestione del modulo investimenti, la modellazione degli ordini e delle sorgenti di pagamento, il calcolo continuo di saldi e posizioni dallo storico, il canale di eventi disaccoppiato e la finestra principale con barra laterale condivisa.

2.4.1 Ordini con Sorgenti di Pagamento Diverse

1. Descrizione del problema

Un acquisto di titoli può essere pagato con un conto di liquidità, con una stablecoin posseduta nello stesso conto di investimento, oppure con fondi esterni; lo stesso vale, al contrario, per il ricavato di una vendita. Ogni combinazione modifica conti diversi, e un ordine può fallire per motivi attesi (fondi insufficienti, quantità non posseduta, prezzo non disponibile). La view deve mostrare all'utente un messaggio preciso per ogni caso, e un ordine fallito non deve modificare nessun conto.

2. Soluzione proposta e alternative

La sorgente di pagamento è modellata tramite un'interfaccia sigillata, `PaymentSource`, con tre implementazioni record: `CashAccountChannel`, `StableCoinPositionChannel` e `NoPaymentChannel`. Anche l'esito di un ordine è un'interfaccia sigillata, `OrderResult`, con un record per ogni successo (che contiene le transazioni create) e uno per ogni errore (che contiene i dati per spiegarlo, ad esempio importo richiesto e disponibile). `DefaultInvestmentController` controlla prima che l'ordine sia eseguibile e solo dopo registra le transazioni; nel codice, la scelta tra le sorgenti è gestita tramite uno `switch` sul tipo con pattern matching, che il compilatore verifica essere esaustivo e completo.

Sono state valutate e scartate tre alternative:

- **Alternativa scartata (Metodi dedicati per combinazione):** Definire un metodo diverso per ogni combinazione di operazione e sorgente avrebbe moltiplicato inutilmente i metodi del controller.
- **Alternativa scartata (Uso di eccezioni):** Ricorrere alle eccezioni per i casi di errore previsto avrebbe appesantito il flusso per situazioni nominali di business, costringendo la view a gestire blocchi `try-catch` multipli.
- **Alternativa scartata (Parametro generico non tipato):** Passare la sorgente come parametro generico (ad esempio un identificativo di conto) avrebbe disperso controlli `instanceof` non verificabili dal compilatore.

Con i tipi sigillati, l'aggiunta di una nuova sorgente di pagamento obbliga il compilatore a segnalare tutti i punti del codice in cui va gestita.

3. Pattern utilizzati e modellazione

Non si tratta di un pattern GoF tradizionale, bensì della modellazione con tipi sigillati (*sealed interfaces*) e `record` (tipi algebrici di dati), che consente di trattare successi ed errori come valori ordinari e di sfruttare il pattern matching per switch con verifica statica di esaustività a compile-time.

2.4.2 Saldo e Posizioni Calcolati dallo Storico

1. Descrizione del problema

Il saldo di un conto di liquidità è la somma dei movimenti; quello di un conto di investimento dipende dai prezzi di mercato del momento, e richiede di calcolare per ogni titolo la quantità posseduta, il prezzo medio di carico e il profitto realizzato con le vendite. In entrambi i casi il saldo deve restare coerente con lo storico anche dopo correzioni e cancellazioni.

2. Soluzione proposta e alternative

Il saldo non viene mai memorizzato in modo ridondante. La classe astratta `AbstractWallet<T extends AbstractTransaction>` contiene lo storico e gestisce tutte le modifiche (aggiunta, rimozione, sostituzione), pubblicando un evento sul bus per ciascuna; il metodo `getBalance()` è astratto e ogni tipo di conto lo calcola a modo suo. Il parametro generico lega ogni conto al proprio tipo di movimento: il compilatore impedisce a tempo di compilazione di aggiungere un acquisto di titoli a un conto di liquidità, senza richiedere controlli a runtime.

`InvestmentAccount` raggruppa i movimenti per titolo, ricava le posizioni (`Position`) con il metodo contabile del costo medio ponderato e delega la richiesta del prezzo corrente a un `CurrencyConverter`. Il convertitore non viene salvato su file JSON: dopo la fase di caricamento i conti vengono ricollegati mediante il metodo `withProvider`. Memorizzare saldo e posizioni direttamente nello stato avrebbe reso le letture marginalmente più rapide, ma avrebbe imposto aggiornamenti a catena a ogni variazione dello storico, con alto rischio di disallineamento dei dati.

3. Pattern utilizzati e reificazione

- **Template Method:** `AbstractWallet` definisce la gestione uniforme dello storico transazioni lasciando astratto il metodo primitivo `getBalance()`, specializzato rispettivamente da `CashAccount` e `InvestmentAccount`.
- **Strategy Pattern:** `InvestmentAccount` usa un'istanza di `CurrencyConverter` come strategia intercambiabile per il recupero delle quotazioni correnti, consentendo nei test automatizzati la sostituzione trasparente con un provider a quotazioni prefissate deterministiche.

2.4.3 Eventi, Tempo dell'Applicazione e Salvataggi Automatici

1. Descrizione del problema

Più componenti dell'applicazione devono reagire dinamicamente ai cambiamenti dello stato informativo: le viste grafiche devono aggiornarsi, i portafogli vanno persistiti su disco, e per il tracciamento del grafico di andamento il profitto/perdita dei conti di investimento va campionato dopo ogni operazione e a intervalli temporali regolari. Il modello di dominio, tuttavia, deve rimanere disaccoppiato dal toolkit JavaFX, che fornisce i timer sul thread dell'interfaccia.

2. Soluzione proposta e alternative

È stato realizzato un bus di eventi disaccoppiato, `MessageBus`: i mittenti pubblicano gli eventi senza conoscere i destinatari, e i sottoscrittori specificano la classe di evento d'interesse ricevendo un token per la deregistrazione. Il tempo dell'applicazione è astratto

tramite l'interfaccia di dominio `AppTicker`; la sua implementazione concreta nella view, `FxAppTicker`, pubblica un `TickEvent` a intervalli regolari.

`PeriodicSnapshotSaver` intercetta sia gli eventi di tick temporale sia le transazioni eseguite, salvando un'istantanea di saldo per ciascun conto di investimento a intervalli controllati tramite `InvestmentsSnapshotService` (su file CSV). Parallelamente, `WalletAutoSaver` ascolta gli eventi di mutazione e persiste i conti su disco; per ogni tipologia di portafoglio riceve una coppia `Supplier` / `Consumer` (lettura e salvataggio), rimanendo completamente agnostico rispetto ai tipi concreti di conto. Prima di questa architettura, la schermata degli investimenti manteneva un timer interno che non veniva rilasciato alla chiusura, provocando duplicazioni ad ogni riapertura.

3. Pattern utilizzati e reificazione

- **Observer Pattern (Publish/Subscribe):** Il `MessageBus` centralizza la mediazione degli eventi applicativi: i portafogli e `FxAppTicker` agiscono da publisher, mentre `PeriodicSnapshotSaver`, `WalletAutoSaver` e le schermate della GUI operano come subscriber.
- **Strategy Pattern:** L'interfaccia `InvestmentsSnapshotService` disaccoppia la strategia di memorizzazione delle serie temporali dei conti titoli dall'algoritmo di campionamento.

2.4.4 Finestra Principale e Barra Laterale Condivisa

1. Descrizione del problema

La finestra principale (`MainViewController`) ospita a turno le schermate dei diversi moduli dell'applicazione. La barra laterale di navigazione è unica e condivisa, ma deve visualizzare la lista dei conti pertinente al modulo attivo (portafogli di liquidità o conti titoli) e inoltrare al controller corrispondente le azioni dell'utente (selezione, creazione, rinomina ed eliminazione). Inoltre, le viste aperte si iscrivono a eventi sul bus, e le iscrizioni devono essere revocate tempestivamente alla chiusura per prevenire memory leak.

2. Soluzione proposta e alternative

La barra laterale (`SideBarViewController`) non mantiene riferimenti diretti ai singoli moduli: interagisce unicamente tramite l'interfaccia formale `SideBarDelegate`, implementata da ciascuna schermata; al cambio di vista, `MainViewController` provvede ad aggiornare il delegate corrente. L'aggiunta di un nuovo modulo con conti dedicati richiede unicamente l'implementazione di tale interfaccia.

Tutti i controller delle viste estendono la classe comune `BaseViewController`, che registra centralmente le sottoscrizioni agli eventi e le rimuove in blocco nel metodo `dispose()`, invocato dalla finestra principale ad ogni transizione di pagina. I controller JavaFX vengono istanziati dalla factory `ViewControllersFactory`, che inietta a tempo di costruzione i servizi necessari. L'alternativa scartata (sviluppare barre laterali separate per ciascun modulo) avrebbe generato una cospicua duplicazione di codice e di stili grafici.

3. Pattern utilizzati e reificazione

- **Strategy Pattern (nella variante Delegate):** L'interfaccia `SideBarDelegate` formalizza la strategia di comunicazione tra la barra laterale e la vista attiva, delegando a quest'ultima l'estrazione dei dati contabili e la gestione dei comandi.

- **Factory Pattern:** La classe `ViewControllersFactory` accentra la creazione e l'inizializzazione dei controller delle viste con iniezione manuale delle dipendenze applicative.

Capitolo 3

Sviluppo

In questo capitolo vengono presentate le attività di testing automatizzato e le caratteristiche avanzate del linguaggio Java impiegate nello sviluppo del sottosistema assegnato.

3.1 Testing Automatizzato

Il progetto adotta la suite **JUnit 5** per garantire l'assenza di regressioni e verificare in modo rigoroso e ripetibile la correttezza della logica applicativa.

Per la parte sviluppata da **Arianna Palmucci**, sono state implementate suite di test unitari e di integrazione esaustive:

- **Motore di Conversione e Precisione Decimale:**
  - `BasicCurrencyConverterTest`: verifica delle conversioni dirette e inverse tra valute fiat.
  - `CurrencyConversionResultTest`: verifica del corretto arrotondamento contabile con `BigDecimal` ed eliminazione degli errori di troncamento.
  - `CurrencyConverterTest`: verifica della coerenza delle conversioni tra asset diversi.
- **Provider di Cambio, Caching e Normalizzazione Dati:**
  - `MockExchangeRateAPITest` e `ExchangeRateAPIClientTest`: validazione della logica di memorizzazione nella cache e correttezza dei dati di fallback.
  - `CurrencyUnitTest` e `FiatCurrencyTest`: verifica rigorosa delle invarianti numeriche e delle proprietà polimorfiche delle valute.
  - `MonetaryInputParserTest`: collaudo esaustivo della sanitizzazione, parsing e gestione unificata dei delimitatori decimali.
  - `AppDataPathResolverTest`: verifica della risoluzione euristica dei percorsi dati e isolamento mediante directory temporanee (`@TempDir`).
- **Simulazioni Bancarie e Commissioni:**
  - `BankTest`, `BankCalculatorTest`, `BankConversionResultTest`, `BankServiceTest`: verifica rigorosa delle commissioni fisse, percentuali e spread bid/ask per istituti di credito simulati.
- **Allerte di Mercato e Watchlist:**

- `CurrencyAlertTest`, `CurrencyAlertServiceTest`, `MarketAlertViewTest`: verifica delle condizioni di innesco delle allerte su soglie di prezzo.
- `WatchListTest`, `WatchlistPairTest`: verifica dell'unicità e della gestione della lista di osservazione.

- **Preferenze, Temi e Memento:**
  - `SettingsTest`, `SettingsSnapshotTest`: verifica della creazione, archiviazione e ripristino degli snapshot di configurazione.
  - `SettingsManagerTest`: test di persistenza e deserializzazione del file JSON delle impostazioni.
  - `ThemeTest`, `WindowPreferencesTest`, `ThemeManagerTest`: verifica della propagazione dei temi e salvataggio delle coordinate geometriche della finestra.

Per la parte sviluppata da **Favour Osaretin Emuwahen**, il collaudo automatizzato si concentra sull'integrità del cruscotto contabile, sulle transazioni e sulle logiche algoritmiche di budget e trend:

- **Dashboard, Facciata e Snapshot Contabile:**
  - `DefaultDashboardFacadeTest`, `DefaultDashboardSnapshotTest`, `DefaultDashboardControll` validazione dell'aggregazione atomica dei dati informativi e coordinamento della vista passiva.
  - `DefaultWalletServiceTest`, `DefaultCategoryServiceTest`, `DefaultFriendLoanSummaryServ` verifica dell'elaborazione analitica dei movimenti e del debito/credito residuo verso amici.

- **Strategie di Budget, Trend Mensili e Specifiche di Filtro:**
  - `BudgetAlertStrategyTest`, `DefaultBudgetMonitorTest`: collaudo delle soglie percentuali e degli stati di allerta (`SAFE`, `WARNING`, `CRITICAL`).
  - `MonthlyInsightTemplateTest`, `DefaultWalletInsightServiceTest`: verifica del calcolo delle variazioni percentuali mese su mese e gestione della divisione per zero.
  - `TransactionSpecificationTest`, `DefaultTransactionHistoryFilterServiceTest`: verifica dei filtri componibili per categoria, data e parole chiave.

- **Costruzione e Validazione Transazioni di Cassa:**
  - `CashTransactionTest`, `CashTransactionBuilderTest`, `CashTransactionFactoryTest`, `CashAccountTest`: verifica dell'immutabilità, della validazione preventiva dei campi e della consistenza dei saldi.

Per la parte sviluppata da **Daniele Zappalà**, le attività di testing automatizzato coprono il modulo investimenti, i contratti di esecuzione degli ordini e le componenti infrastrutturali:

- **Investimenti e Rendicontazione Portafoglio Titoli:**
  - `InvestmentAccountTest`: verifica del calcolo delle posizioni contabili, del prezzo medio ponderato di carico, del profitto/perdita non realizzato e del profitto realizzato a fronte di vendite parziali.

  - `DefaultInvestmentControllerTest`: collaudo esaustivo degli ordini di acquisto, vendita e trasferimento con tutte le sorgenti di pagamento (`PaymentSource`), gestendo sia gli esiti nominali sia le condizioni anomale (fondi insufficienti, disponibilità titoli carente, conversioni tra valute diverse).
  - `InvestmentAccountPersistenceTest`: test di persistenza e ricaricamento da file JSON con conservazione dell'identificativo UUID.

- **Campionamento Temporale e Fonti Dati:**
  - `PeriodicSnapshotSaverTest`: verifica del salvataggio periodico delle istantanee temporali mediante ticker simulato deterministico.
  - `CryptoPriceBatchTest`: test di deserializzazione delle quotazioni da payload memorizzati, compresa la gestione di rate limit e quote esaurite.

Tutti i test vengono eseguiti automaticamente dal task `./gradlew check`. Il codice rispetta rigorosamente i controlli statici di Checkstyle, PMD e SpotBugs con zero violazioni.

3.2 Note di Sviluppo

In questa sezione vengono documentate le caratteristiche avanzate del linguaggio Java (versione 21 LTS) e delle librerie applicative impiegate nei rispettivi moduli, evidenziando le scelte implementative che hanno garantito modularità, robustezza e concisione del codice.

3.2.1 Favour Osaretin Emuwahen

Nello sviluppo del sottosistema Dashboard e Contabilità sono state valorizzate le potenzialità introdotte dalle recenti versioni di Java, coniugando paradigmi orientati agli oggetti e funzionali:

- **Gestione Idiomatica di `Optional<T>` e Validazione delle Invarianti di Dominio:** Eliminazione dei valori nulli dall'API pubblica mediante l'impiego pervasivo di `Optional<T>` per la modellazione dei dati accessori di prestito fiduciario (`friendLoanId`, `friendName`) in `CashTransaction` e dei filtri di ricerca in `TransactionFilterCriteria`. La consistenza dello stato contabile viene garantita da rigidi controlli di validazione nel costruttore (*fail-fast*), che rifiutano combinazioni spurie di parametri (es. identificativo di prestito valorizzato senza il relativo nominativo).

  Riferimento: https://github.com/leoI0II/OOP25-UniBodget/blob/1f46f6c0064714ab0f900d9928433eef1f5d965a/app/src/main/java/it/unibo/unibodget/model/transactions/base/CashTransaction.java#L80-L98

- **Fluent Builder con Supporto Prototype Copying (*Copy-on-Write*):** Implementazione del Builder pattern per `CashTransaction` equipaggiato sia con metodi di assemblaggio fluente a tipizzazione stretta, sia con un costruttore di copia specializzato (`builder(CashTransaction prototype)`). Tale accorgimento abilita la derivazione non distruttiva di nuove transazioni rettificate a partire da entità preesistenti, preservando al contempo la rigorosa immutabilità dei movimenti registrati nel portafoglio.

  Riferimento: https://github.com/leoI0II/OOP25-UniBodget/blob/1f46f6c0064714ab0f900d9928433eef1f5d965a/app/src/main/java/it/unibo/unibodget/model/transactions/base/CashTransaction.java#L208-L258

- **Interfacce Funzionali e Generici Componibili (`Predicate<T>`):** Reificazione dello Specification Pattern tramite un'interfaccia che estende `Predicate<CashTransaction>` e fornisce combinatori logici di default (`and`, `or`, `not`) basati su lambda, consentendo l'assemblaggio dinamico e dichiarativo di filtri complessi.

  Riferimento: https://github.com/leoI0II/OOP25-UniBodget/blob/1f46f6c0064714ab0f900d9928433eef1f5d965a/app/src/main/java/it/unibo/unibodget/model/dashboard/specification/TransactionSpecification.java#L17-L63

- **Pattern Matching per Switch e Record Patterns (Java 21):** Impiego del pattern matching per switch con decostruzione contestuale dei record per gestire le diverse modalità di regolamento economico in `NewTransactionDialog`, azzerando cast espliciti e variabili di appoggio temporanee.

  Riferimento: https://github.com/leoI0II/OOP25-UniBodget/blob/1f46f6c0064714ab0f900d9928433eef1f5d965a/app/src/main/java/it/unibo/unibodget/view/dashboard/impl/NewTransactionDialog.java#L587-L594

- **Record Immutabili per il Trasferimento Dati (*Data Transfer Objects*):** Modellazione dello stato di presentazione della UI (`DashboardViewState`, `HeaderCardState`, `BalanceCardState`) e degli esiti elaborati dai servizi di calcolo (`BudgetEvaluationResult`, `WalletInsight`) tramite record nativi immutabili, eliminando codice boilerplate, garantendo consistenza atomica e scongiurando side-effect tra strati architetturali.

  Riferimento: https://github.com/leoI0II/OOP25-UniBodget/blob/1f46f6c0064714ab0f900d9928433eef1f5d965a/app/src/main/java/it/unibo/unibodget/view/dashboard/state/DashboardViewState.java#L33-L72

- **Stream API Avanzate, Raggruppamento Funzionale e Switch Expressions:** Elaborazione analitica dei movimenti finanziari tramite pipeline funzionali con operatori intermedi e terminali (`.filter()`, `.map()`, `.reduce()`) per il calcolo cumulativo dei saldi e raggruppamento per tipologia di spesa tramite `Collectors.groupingBy()` in `DefaultCategoryService`. In `DefaultTransactionHistoryFilterService`, l'ordinamento multicriterio combina `Comparator.comparing(...).thenComparing(...)` e `Comparator.nullsLast` con una *switch expression* esaustiva su enum, restituendo una collezione non modificabile tramite `Stream.toList()`.

  Riferimento: https://github.com/leoI0II/OOP25-UniBodget/blob/1f46f6c0064714ab0f900d9928433eef1f5d965a/app/src/main/java/it/unibo/unibodget/model/dashboard/impl/DefaultTransactionHistoryFilterService.java#L64-L95

- **Aritmetica Decimale Esatta con `BigDecimal`:** Utilizzo pervasivo di `BigDecimal` con scala esplicita e arrotondamento controllato (`RoundingMode.HALF_UP`) per tutte le grandezze monetarie e percentuali, eliminando alla radice le imprecisioni dell'aritmetica floating-point IEEE 754.

  Riferimento: https://github.com/leoI0II/OOP25-UniBodget/blob/1f46f6c0064714ab0f900d9928433eef1f5d965a/app/src/main/java/it/unibo/unibodget/model/dashboard/impl/AbstractMonthlyInsightTemplate.java#L32-L72

3.2.2 Arianna Palmucci

Nello sviluppo del modulo di gestione delle valute, quotazioni, persistenza JSON, impostazioni e tematizzazione dinamica sono state impiegate le seguenti caratteristiche avanzate dell'ecosistema Java 21:

- **Networking Moderno con HTTP/2 e Client Asincrono (`java.net.http.*`):** Interrogazione dei servizi RESTful di cambio valuta tramite `HttpClient` e `HttpRequest`, configurati con timeout espliciti (`Duration`), gestione nativa del protocollo HTTP/2 e logiche di fallback trasparente sui dati in cache o predefiniti in assenza di rete.

  Riferimento: https://github.com/leoI0II/OOP25-UniBodget/blob/1f46f6c0064714ab0f900d9928433eef1f5d965a/app/src/main/java/it/unibo/unibodget/model/currency/api/ExchangeRateAPIClient.java#L133-L157

- **Serializzazione JSON Polimorfica e Moduli Custom (Jackson):** Configurazione avanzata di `ObjectMapper` in `PersistenceJacksonConfig` con il modulo `JavaTimeModule` e deserializzatori ad hoc per gestire la persistenza delle preferenze, consentendo la ricostruzione polimorfica delle istanze di `CurrencyUnit` tramite factory method senza violare l'incapsulamento.

  Riferimento: https://github.com/leoI0II/OOP25-UniBodget/blob/1f46f6c0064714ab0f900d9928433eef1f5d965a/app/src/main/java/it/unibo/unibodget/persistency/parser/impl/PersistenceJacksonConfig.java#L54-L80

- **Gestione Resiliente del File System con Java NIO.2 (`java.nio.file.*`):** Risoluzione centralizzata dei percorsi dati in `AppDataPathResolver`, con creazione automatica delle cartelle di lavoro, gestione atomica dei file e totale indipendenza dal sistema operativo ospitante (macOS, Windows, Linux).

  Riferimento: https://github.com/leoI0II/OOP25-UniBodget/blob/1f46f6c0064714ab0f900d9928433eef1f5d965a/app/src/main/java/it/unibo/unibodget/persistency/AppDataPathResolver.java#L159-L283

- **Robustezza Funzionale e Aritmetica con `Optional` e `BigDecimal`:** Adozione sistematica di `Optional<BigDecimal>` in `MonetaryInputParser` per gestire stringhe monetarie eterogenee (delimitatori punto/virgola) azzerando i riferimenti `null`, affiancata da calcoli di conversione monetaria e commissioni bancarie su `BigDecimal` con `RoundingMode.HALF_UP`.

  Riferimento: https://github.com/leoI0II/OOP25-UniBodget/blob/1f46f6c0064714ab0f900d9928433eef1f5d965a/app/src/main/java/it/unibo/unibodget/model/currency/MonetaryInputParser.java#L46-L88

- **Callback Funzionali Reattive e Strutture Dati Immutabili:** Impiego dell'interfaccia funzionale `Consumer<Theme>` in `ThemeManager` per registrare callback invocate alla commutazione del tema visivo a caldo, e incapsulamento rigoroso dei risultati di conversione tramite la classe immutabile `CurrencyConversionResult` (affiancata nel modulo dai record `Asset` per gli importi), garantendo assenza di side-effect.

  Riferimento: https://github.com/leoI0II/OOP25-UniBodget/blob/1f46f6c0064714ab0f900d9928433eef1f5d965a/app/src/main/java/it/unibo/unibodget/model/currency/CurrencyConversionResult.java#L10-L37

3.2.3 Daniele Zappalà

Nello sviluppo del modulo investimenti, del canale di eventi disaccoppiato e dell'infrastruttura condivisa sono state adottate le seguenti caratteristiche avanzate del linguaggio Java 21:

- **Tipi Sigillati e Pattern Matching per Switch:** Modellazione gerarchica delle sorgenti di pagamento e degli esiti degli ordini tramite interfacce sigillate (`PaymentSource`, `OrderResult`) e record immutabili, gestiti nei controller mediante switch esaustivi con decostruzione dei tipi verificati a tempo di compilazione.

  Riferimento: https://github.com/leoI0II/OOP25-UniBodget/blob/1f46f6c0064714ab0f900d9928433eef1f5d965a/app/src/main/java/it/unibo/unibodget/model/investment/PaymentSource.java#L23-L52

- **Generici con Type Token e Programmazione Funzionale (`MessageBus`):** Implementazione del bus applicativo disaccoppiato tramite un metodo generico limitato (`<T extends MessageEvent>`) basato su type token (`Class<T>`) e consumatori funzionali (`Consumer<T>`), garantendo type-safety a runtime nella distribuzione degli eventi.

  Riferimento: https://github.com/leoI0II/OOP25-UniBodget/blob/1f46f6c0064714ab0f900d9928433eef1f5d965a/app/src/main/java/it/unibo/unibodget/model/utils/MessageBus.java#L59-L63

- **Stream API con `Collectors.groupingBy` e Riduzioni Complesse:** Calcolo delle posizioni di portafoglio aperte e determinazione contabile del costo medio ponderato aggregando le transazioni storiche tramite raggruppamento funzionale con `Collectors.groupingBy` in `InvestmentAccount`.

  Riferimento: https://github.com/leoI0II/OOP25-UniBodget/blob/1f46f6c0064714ab0f900d9928433eef1f5d965a/app/src/main/java/it/unibo/unibodget/model/wallet/InvestmentAccount.java#L146-L154

- **Astrazione del Tempo Applicativo con JavaFX `Timeline`:** Separazione netta della sorgente temporale mediante l'astrazione di dominio `AppTicker`, la cui implementazione grafica sfrutta una `Timeline` JavaFX per scandire i cicli periodici di snapshot e aggiornamento.

  Riferimento: https://github.com/leoI0II/OOP25-UniBodget/blob/1f46f6c0064714ab0f900d9928433eef1f5d965a/app/src/main/java/it/unibo/unibodget/view/utils/FxAppTicker.java#L29-L39

- **I/O Tabellare con Apache Commons CSV e Salvataggio Automatico:** Serializzazione e storicizzazione delle istantanee patrimoniali su file CSV tramite la libreria Commons CSV, integrata con un motore di salvataggio automatico (`WalletAutoSaver`) basato su `Supplier` e `Consumer`.

  Riferimento: https://github.com/leoI0II/OOP25-UniBodget/blob/1f46f6c0064714ab0f900d9928433eef1f5d965a/app/src/main/java/it/unibo/unibodget/persistency/WalletAutoSaver.java#L59-L65

- **Reflection e Inizializzazione Dinamica dei Controller FXML:** Istanziazione centralizzata dei controller di vista in `ViewControllersFactory` tramite reflection controllata per iniettare programmaticamente le dipendenze applicative nei moduli caricati a runtime.

Riferimento: https://github.com/leoI0II/OOP25-UniBodget/blob/1f46f6c0064714ab0f900d9928433eef1f5d965a/app/src/main/java/it/unibo/unibodget/view/main/ViewControllersFactory.java#L64-L91

Capitolo 4

Commenti finali

4.1 Autovalutazione e lavori futuri

4.1.1 Favour Osaretin Emuwahen

Nel corso del progetto UniBodget mi sono dedicato principalmente al nucleo contabile e alla schermata della Dashboard: la gestione dei conti di liquidità (`CashAccount`), la creazione e validazione rigorosa delle transazioni di cassa, il monitoraggio delle soglie di budget, l'elaborazione statistica dei trend mensili e il motore di ricerca e filtraggio avanzato dei movimenti.

Punti di forza del contributo:

- **Architettura modulare ed estendibile:** L'adozione mirata di pattern comportamentali (Template Method per i trend, Specification per i filtri, Strategy per le allerte del budget) ha evitato catene di controlli condizionali imperativi, permettendo di introdurre nuovi criteri di calcolo o regole di business in modo pulito e senza alterare il codice esistente.
- **Integrità contabile e validazione preventiva:** L'uso del Builder per `CashTransaction` impedisce alla radice la creazione di transazioni in stati parziali o incoerenti. Per tutti i calcoli su saldi e percentuali abbiamo scelto l'uso pervasivo di `BigDecimal` con scala esplicita, preservando l'esattezza contabile al centesimo ed eliminando i tipici errori di approssimazione floating-point.
- **Affidabilità della suite di test:** Ho sviluppato una suite approfondita di test unitari con JUnit 5 per verificare sia i casi nominali sia le condizioni al contorno più insidiose (come la divisione per zero nel calcolo dei trend in assenza di spese nel mese precedente o l'intervallo temporale invertito nei filtri).

Punti di debolezza ed elementi migliorabili:

- **Calcoli sincroni sulla dashboard:** Il ricalcolo dei totali patrimoniali e il filtraggio avvengono attualmente sul thread chiamante; con moli di transazioni molto voluminose, delegare l'elaborazione a worker asincroni garantirebbe una reattività dell'interfaccia ancora più fluida.
- **Conversione multivaluta diretta nel cruscotto:** La dashboard mostra i dati nella valuta base del portafoglio corrente; integrare direttamente nel cruscotto

una visualizzazione a saldo convertito istantaneo su qualsiasi divisa internazionale arricchirebbe l'esperienza d'uso.

Ruolo all'interno del gruppo:

Lavorare in team è stata un'esperienza costruttiva: con Arianna e Daniele abbiamo concordato fin dalle prime fasi i contratti dei DTO condivisi tramite record Java. Questa chiara definizione dei confini architetturali ha permesso a ciascuno di noi di procedere in autonomia e in parallelo sui rispettivi moduli (convertitore valute e investimenti), integrandoci poi con naturalezza nel flusso dei conti senza bloccarci a vicenda.

Sviluppi futuri specifici del modulo:

1. **Budgeting multi-categoria e analisi predittiva del burn-rate:** Possibilità di impostare tetti di spesa indipendenti per singole categorie merceologiche (es. alimentari, trasporti, tempo libero) e introduzione di un algoritmo di stima basato sul ritmo di spesa rilevato a metà mese (*burn-rate*) per allertare preventivamente l'utente prima del superamento del limite.

2. **Gestione delle spese ricorrenti e abbonamenti periodici:** Supporto nativo per transazioni a scadenza programmata (affitto, utenze, abbonamenti a servizi digitali) con generazione automatica dei movimenti contabili o promemoria visivi nella dashboard all'avvicinarsi della data prefissata.

3. **Evoluzione del modulo prestiti in un registro dedicato (*Loan Ledger*):** Trasformazione dell'attuale tracciamento basato su metadati in una vera entità aggregata autonoma capace di monitorare scadenze, promesse di pagamento e piani di rientro a rate parziali con estinzione progressiva del saldo fiduciario.

4. **Esportazione contabile avanzata e reportistica (CSV, PDF, Excel):** Generazione automatica di estratti conto mensili e riepiloghi patrimoniali esportabili, completi di prospetti grafici aggregati per l'archiviazione e rendicontazione personale o familiare.

5. **Elaborazione asincrona dei dati della Dashboard:** Spostamento dell'orchestrazione sincrona svolta da `DefaultDashboardFacade` su worker thread in background tramite le API di concorrenza di JavaFX (`javafx.concurrent.Task`), garantendo una reattività ottimale anche a fronte di cronologie con decine di migliaia di transazioni.

4.1.2 Arianna Palmucci

All'interno del progetto mi sono occupata del modulo Valute e Mercato: il motore di conversione monetaria, l'integrazione del client di rete per i tassi remoti con caching e fallback locale, il sistema di notifiche e watchlist, la persistenza su file JSON delle preferenze utente, l'infrastruttura di tematizzazione dinamica a runtime e la normalizzazione centralizzata dell'input numerico.

Punti di forza del contributo:

- **Resilienza alle anomalie di rete e continuità d'uso:** Uno degli aspetti su cui ho lavorato con maggiore cura è stato garantire che l'applicazione non andasse mai

in blocco in assenza di connessione. Con il meccanismo di cache controllata da TTL e il fallback automatico su mock deterministico o sugli ultimi dati validi memorizzati, il convertitore continua a rispondere puntualmente anche a fronte di disconnessioni o rate-limit dei server esterni.

- **Normalizzazione centralizzata dell'input:** La classe `MonetaryInputParser` ha risolto alla radice le eterogeneità di digitazione dell'utente, accettando indifferentemente virgola e punto decimale ed eliminando spazi e apici. L'uso di `Optional<BigDecimal>` restituisce un feedback chiaro ed elegante ai form, evitando che eccezioni non gestite compromettano l'esperienza grafica.
- **Modularità della vista e tematizzazione a caldo:** La schermata del convertitore è strutturata in widget autonomi facilmente riposizionabili. Le impostazioni dell'utente persistono in formato JSON con storico reversibile (ispirato al pattern Memento), e la commutazione tra tema chiaro e scuro avviene istantaneamente a runtime senza dover riavviare l'applicazione.

Punti di debolezza ed elementi migliorabili:

- **Singolo provider di rete primario:** Il client per le valute fiat si appoggia principalmente a un unico servizio remoto; una catena di provider multipli con fallback automatico a cascata avrebbe offerto una ridondanza ancora superiore.
- **Caricamento on-demand dei grafici storici:** Il recupero dei dati storici su archi temporali estesi avviene all'apertura del dialogo; anticipare il pre-caricamento in background ridurrebbe ulteriormente la latenza percepita dall'utente.

Ruolo all'interno del gruppo:

Nel lavoro di squadra mi sono concentrata sulla definizione delle basi multivaluta comuni (`CurrencyUnit`) e sulla coerenza visiva dell'intera suite attraverso l'architettura dei temi CSS. Ho inoltre collaborato alla definizione del `MessageBus` e alla corretta gestione delle finestre popup e modali sopraelevate (`SettingsPopupFX` e grafici di andamento), assicurandomi che ogni dialogo secondario si aprisse come una vera finestra contestuale senza invadere lo schermo intero.

Sviluppi futuri specifici del modulo:

1. **Provider di cambio multipli con fallback a cascata:** Interrogazione in sequenza di più fonti web pubbliche (es. BCE, piattaforme crypto) per garantire quotazioni aggiornate anche a fronte di rate limit o indisponibilità temporanee.
2. **Comparatore avanzato delle commissioni bancarie:** Algoritmo di confronto multicriterio tra le banche censite per consigliare all'utente l'istituto o il canale più conveniente in base alla valuta e all'importo scambiato.
3. **Basket valutario personalizzato e indici sintetici:** Possibilità per l'utente di definire panieri monetari su misura e indici compositi (es. combinando valute fiat e criptovalute con pesi percentuali) per monitorare in tempo reale il valore ponderato del proprio portafoglio internazionale.

4.1.3 Daniele Zappalà

Nel progetto mi sono dedicato alla progettazione e implementazione del modulo investimenti (`InvestmentAccount`, gestione degli ordini di acquisto e vendita con molteplici sorgenti di pagamento, calcolo delle posizioni aperte e del profitto/perdita), oltre ad aver realizzato la finestra principale (`MainViewController`) con la barra laterale condivisa a navigazione delegata, il bus di eventi disaccoppiato (`MessageBus`), l'astrazione temporale (`AppTicker`) e il salvataggio automatico periodico.

Punti di forza del contributo:

- **Espressività dei tipi sigillati e pattern matching:** Modellare sorgenti ed esiti delle transazioni tramite interfacce sigillate (`PaymentSource`, `OrderResult`) e record ha reso il flusso operativo estremamente leggibile e sicuro: il compilatore verifica l'esaustività degli switch, eliminando la necessità di ricorrere a eccezioni per gestire esiti nominali di business (come fondi insufficienti).
- **Integrità contabile derivata dallo storico:** La decisione di non salvare saldo e posizioni come campi mutabili volatili, ma di ricavarli in modo trasparente e dinamico dallo storico delle operazioni con il metodo a costo medio ponderato, ha azzerato i rischi di disallineamento matematico dei dati anche a seguito di modifiche o cancellazioni retroattive.
- **Indipendenza del modello dal framework grafico:** La logica finanziaria e la scansione del tempo sono del tutto slegate da JavaFX, rendendo il core applicativo pienamente collaudabile in modo deterministico nei test automatici.

Punti di debolezza ed elementi migliorabili:

- **Canale eventi globale:** Il `MessageBus` adotta un pattern ad accesso statico: una soluzione immediata ed efficace per connettere viste e controller disaccoppiati, ma che richiede particolare pulizia nei test unitari per non condividere accidentalmente subscriber tra suite diverse; un'iniezione esplicita per istanza avrebbe garantito un isolamento ancora più rigoroso.
- **Ricalcolo retroattivo degli snapshot storici:** Le serie storiche salvate periodicamente su file CSV registrano lo stato patrimoniale al momento del campionamento; modificare ordini passati non rigenera in automatico i vecchi punti del grafico.

Ruolo all'interno del gruppo:

Oltre allo sviluppo della sezione investimenti, ho curato la cornice principale dell'interfaccia con la barra laterale unificata in cui convergono le schermate dei vari moduli. Mi sono inoltre fatto carico della gestione del canale eventi comune e ho coordinato buona parte dell'integrazione del codice su Git, facilitando la risoluzione dei conflitti e la convergenza dei rami di lavoro.

Sviluppi futuri specifici del modulo:

1. **Analitica temporale avanzata dei grafici:** Selezione dinamica del timeframe (settimanale, mensile, annuale) e visualizzazione della curva di rendimento per ogni singolo strumento in portafoglio.
2. **Rigenerazione retroattiva degli snapshot:** Algoritmo di ricalcolo automatico della serie storica CSV per riallineare i dati in caso di modifiche o eliminazioni di ordini passati.

4.1.4 Sviluppi Futuri Comuni

Dal confronto collegiale tra i membri del gruppo sono emerse alcune direttrici evolutive di respiro sistemico, che rappresentano il naturale proseguimento dell'architettura condivisa qualora il software venisse esteso per un impiego su larga scala:

1. **Persistenza Remota e Sincronizzazione Cloud Multi-dispositivo:** L'attuale memorizzazione locale su file JSON e CSV garantisce privacy e funzionamento offline. La sua naturale evoluzione consiste nell'integrazione di un database relazionale (es. PostgreSQL) o di un connettore cloud cifrato, consentendo all'utente di sincronizzare in modo trasparente conti, transazioni e impostazioni su più dispositivi, con supporto a backup automatici e ripristino di emergenza.
2. **Pipeline Concorrente e Reattività Asincrona Non Bloccante:** Attualmente il caricamento dei dati di rete e i calcoli aggregati avvengono principalmente in modo sincrono sul thread chiamante. L'adozione sistematica di un'infrastruttura asincrona basata su `CompletableFuture` e thread worker in background permetterebbe di scaricare quotazioni remote, elaborare statistiche complesse e aggiornare gli snapshot patrimoniali in parallelo, mantenendo l'interfaccia JavaFX costantemente fluida e reattiva anche con moli di dati storici considerevoli.
3. **Apertura ad Architetture Web e Mobile:** La netta separazione architetturale tra il Model (completamente disaccoppiato dalla GUI) e i View Controller rende il nucleo logico di UniBodget immediatamente riusabile. Un'evoluzione strategica prevede l'esposizione di API RESTful attorno ai servizi di dominio esistenti, consentendo di affiancare all'attuale applicazione desktop un portale web responsivo e un'applicazione mobile nativa, condividendo le medesime regole di business per la gestione contabile, valutaria e degli investimenti.

4.2 Difficoltà Incontrate e Commenti per i Docenti

Nel corso del progetto, il team ha affrontato e superato diverse difficoltà significative:

- **Conciliazione tra JavaFX e Rigorosa Analisi Statica (QA):** L'integrazione del toolkit grafico JavaFX con le severe regole di analisi statica imposte dal plugin del Prof. Pianini (`gradle-java-qa`, comprendente Checkstyle, PMD e SpotBugs) ha richiesto un impegno notevole. In particolare, la gestione delle classi interne anonime, la rimozione sistematica di costanti magiche di layout e la corretta giustificazione dei pattern di esposizione di riferimenti per i nodi grafici hanno reso necessario uno studio approfondito della progettazione ad oggetti.
- **Eliminazione dei Numeri Magici e Token Tipografici:** Rendere l'interfaccia JavaFX dinamica e gradevole rispettando il vincolo di zero numeri magici ha imposto l'estrazione meticolosa di ogni spaziatura, coordinata grafica e raggio di curvatura in classi costanti di utilità visiva.
- **Aritmetica e Precisione Finanziaria:** Garantire la coerenza monetaria tra saldi di cassa, quote azionarie frazionate e percentuali di cambio ha imposto l'adozione pervasiva di `BigDecimal`, prestando massima attenzione alle scale di divisione e alla gestione delle divisioni per zero nei trend statistici.
- **Lavoro Cooperativo e Versionamento con Git:** La suddivisione del lavoro su branch paralleli e la convergenza su classi condivise (come `MainViewController` e i modelli di portafoglio) hanno consolidato le competenze del gruppo nell'uso professionale del controllo di versione, nella risoluzione dei conflitti e nella revisione del codice tramite pull request.

Appendice A

Guida Utente

Questa appendice illustra in modo operativo le funzionalità di **UniBodget**, guidando l'utente attraverso l'esecuzione del software e le principali operazioni finanziarie.

Avvio dell'Applicazione

Per compilare ed eseguire l'applicazione desktop, è sufficiente posizionarsi nella cartella principale del progetto ed eseguire tramite terminale il comando Gradle:

```
./gradlew run
```

In alternativa è possibile creare il pacchetto eseguibile autocontenuto con `./gradlew jar` e avviarlo con `java -jar UniBodget.jar`. Il runtime JavaFX provvederà a caricare le preferenze utente persistite, ripristinare i conti, le transazioni e le impostazioni, e visualizzare la finestra principale dell'applicazione, suddivisa tra la barra laterale di navigazione (*Sidebar*) e l'area centrale, impostata di default sulla Dashboard.

Per il recupero delle quotazioni di mercato è richiesta una connessione a Internet; per l'accesso ai titoli azionari tramite provider esterno è supportata una chiave API personale configurabile tramite la variabile d'ambiente `ALPHA_VANTAGE_API_KEY`. Le serie temporali delle istantanee del portafoglio titoli vengono memorizzate all'interno della cartella di persistenza (`~/.unibodget/snapshots` o `data/`).

Panoramica della Dashboard

La schermata principale della Dashboard è organizzata in aree logiche ad alta densità informativa:

1. **Barra superiore di intestazione:** Mostra il titolo della schermata, il selettore del portafoglio attivo (*Cash Account*) tramite menu a tendina e il pulsante `"New Wallet"` per la creazione di un nuovo portafoglio.

2. **Scheda Patrimonio e Saldo:** Espone a caratteri cubitali il saldo contabile totale del conto attivo, formattato nella divisa di riferimento (es. EUR, USD).

3. **Riepilogo Trend Mensili (Monthly Insights):** Tre schede comparative che espongono in tempo reale il confronto rispetto al mese precedente per:

    - *Risparmi netti:* Differenza tra entrate e uscite con indicazione del delta percentuale.

- *Uscite totali:* Somma delle spese del mese con badge cromatico di tendenza.
- *Entrate totali:* Totale dei ricavi incassati rispetto al mese antecedente.

4. **Sezione Monitoraggio Budget:** Indicatore grafico circolare che riporta la percentuale di budget consumata, accompagnata da un badge cromatico:
   - *Verde (SAFE):* Spese ampiamente al di sotto della soglia limite.
   - *Giallo/Arancio (WARNING):* Superamento della soglia percentuale di preallarme (es. 80%).
   - *Rosso (CRITICAL):* Saturazione o superamento del 100% del limite prefissato.

   Accanto all'indicatore è presente il pulsante di modifica rapida per aggiornare il limite massimo e la soglia percentuale.
5. **Grafico Ripartizione Spese:** Diagramma circolare interattivo (*PieChart*) con tooltip dinamici che scompone le uscite per voce di costo (Alimentari, Affitto, Trasporti, Svago, ecc.).
6. **Tabella Cronologia Movimenti:** Prospetto analitico con colonne per Data, Descrizione, Categoria, Importo monetario, Tipologia e pulsante di cancellazione rapida.

Operazioni Guidate

Inserimento di una Nuova Transazione

1. Premere il pulsante `"+ New Transaction"` situato nella schermata della Dashboard.
2. Selezionare la Tipologia contabile desiderata (*Expense*, *Income*, *Friend Loan*).
3. Inserire l'importo monetario (il sistema valida automaticamente cifre, virgole e punti decimali).
4. Selezionare la categoria semantica associata e specificare la data tramite il calendario integrato.
5. Inserire una causale descrittiva e note facoltative.
6. Se si registra un prestito informale (*Friend Loan*), specificare obbligatoriamente il nominativo dell'amico oppure selezionare un prestito pregresso per registrarne la restituzione.
7. Premere `"Save Transaction"`. Il saldo, i grafici e il budget si aggiorneranno istantaneamente in modo atomico.

Filtraggio Avanzato della Cronologia

1. Impostare l'intervallo temporale tramite i selettori di data iniziale e finale.
2. Selezionare un filtro per tipologia di movimento o per categoria dal rispettivo menu a tendina.
3. Digitare una parola chiave nel campo di ricerca testuale (la ricerca opera in modalità case-insensitive su descrizione e note).

4. L'ordinamento può essere commutato a piacere (per data o per importo).

5. Cliccando su `"Export CSV"`, la cronologia filtrata viene esportata su un file tabellare conforme allo standard RFC 4180.

Portfolio e Gestione degli Investimenti

La schermata **Portfolio** consente di monitorare i conti titoli, visualizzare l'andamento del patrimonio investito ed eseguire ordini di borsa:

1. **Indicatori Patrimoniali del Conto:** La parte superiore mostra il valore di mercato totale del conto, il profitto/perdita complessivo (sia in valore monetario sia in percentuale), il costo storico di carico e i titoli che registrano il rendimento migliore e peggiore.

2. **Grafici di Performance e Asset Allocation:** Il grafico ad area/linea temporale (*Performance*) traccia l'andamento del profitto/perdita nel tempo campionando le istantanee; due grafici a torta visualizzano la ripartizione del portafoglio tra i diversi asset per valore monetario e per quantità posseduta.

3. **Emissione di Ordini (`"+ Add transaction"`):** Premendo il pulsante dedicato si apre una finestra di dialogo modale articolata su tre schede:

   - *BUY (Acquisto):* Permette di selezionare il titolo (il prezzo corrente viene precompilato automaticamente), la quantità, le commissioni e la sorgente di pagamento: un conto di liquidità (*Cash account*), saldo in stablecoin del conto stesso (*Stable coin*) o fondo esterno (*No payment*). Il totale dovuto si aggiorna dinamicamente durante l'inserimento.
   - *SELL (Vendita):* Analogo all'acquisto, mette a disposizione il pulsante rapido *MAX* per liquidare l'intera posizione; il controvalore netto ricavato viene accreditato sulla sorgente selezionata.
   - *TRANSFER (Trasferimento):* Sposta quote di un asset verso un altro conto di investimento senza generare compravendite fittizie.

   Se un ordine non può essere eseguito (ad esempio per fondi insufficienti, quantità da vendere superiore a quella posseduta o assenza di quotazione), l'applicazione mostra un messaggio diagnostico esplicativo e l'ordine viene respinto senza alterare alcun conto.

4. **Tabelle Posizioni e Movimenti:** Le tabelle *Assets* e *Transactions* elencano le posizioni aperte e lo storico dei movimenti eseguiti, con supporto all'ordinamento interattivo per colonna. Il pulsante *Export CSV* consente di esportare l'intero storico su foglio di calcolo.

Convertitore di Valute e Servizi di Mercato

La sezione **Currency Converter** mette a disposizione una suite completa di strumenti per il calcolo dei cambi, il monitoraggio delle quotazioni e l'analisi dei costi di conversione:

1. **Inserimento dell'Importo e Normalizzazione:** Nel campo `"Amount"` l'utente può inserire la cifra da convertire. Il parser accetta indifferentemente sia la virgola sia il punto come separatore decimale, normalizzando automaticamente il valore inserito senza generare errori di formattazione.

2. **Selezione delle Divise (From e To):** Tramite due appositi menu a tendina è possibile selezionare la valuta di origine e quella di destinazione, attingendo dall'intero paniere supportato (valute tradizionali come EUR, USD, GBP, JPY, criptovalute come BTC, ETH e titoli azionari come AAPL, MSFT, GOOGL).

3. **Inversione Istantanea delle Valute (Pulsante Swap ↕):** Posizionato tra i due selettori valutari, il pulsante `"Swap"` inverte con un solo clic la valuta sorgente e la valuta target, permettendo di consultare il tasso inverso senza dover reimpostare manualmente i campi.

4. **Calcolo della Conversione Reale (`"Convert"`):** Cliccando su `"Convert"`, il sistema interroga il motore di calcolo basato su tassi di mercato aggiornati (con caching resiliente e fallback in memoria) e mostra il risultato netto calcolato (`"Net Balance Result"`) con arrotondamento bancario conforme agli standard finanziari.

5. **Monitoraggio Soglie e Allerte di Mercato (`"Threshold"`):** È possibile specificare una soglia numerica nel campo `"Threshold"` e impostare la direzione desiderata (*Above* se si attende un rialzo sopra la soglia, oppure *Below* se si monitora un ribasso). L'applicazione controllerà il tasso corrente e notificherà tempestivamente l'utente all'attivazione della condizione.

6. **Simulatore delle Spese Bancarie (`"Bank Fees"`):** Nel riquadro dedicato, l'utente può selezionare un istituto bancario convenzionato e premere `"Calculate Bank Fees"`. Il simulatore visualizza lo spread commerciale percentuale applicato dalla banca e le commissioni fisse di transazione, evidenziando la differenza netta tra il cambio puro di mercato e la somma effettivamente ricevuta.

7. **Gestione della Watchlist Personale (`"My Watchlist"`):** Premendo il pulsante `"Save couple in watchlist"`, la coppia valutaria attualmente impostata viene salvata nella lista dei preferiti a lato dello schermo. Le coppie monitorate rimangono memorizzate tra un avvio e l'altro, e possono essere consultate o eliminate in qualsiasi momento con un semplice clic.

8. **Grafico Storico Interattivo (`"Show Chart"`):** Cliccando su `"Show Chart"` nella barra superiore, si apre una finestra popup dedicata contenente un grafico ad area/linea temporale. Selezionando la coppia di interesse e premendo `"Load Chart"`, viene tracciata la serie storica delle quotazioni recenti con l'indicazione visiva delle soglie di allerta.

Chiusura dell'Applicazione

Sulla barra superiore dell'interfaccia è presente il pulsante `"Close"`, che consente all'utente di uscire e chiudere l'applicazione in modo semplice e immediato in qualsiasi momento.

Impostazioni di Sistema e Personalizzazione (`Settings`)

Cliccando sul pulsante `"Settings"` posizionato nella barra laterale o nella scheda dedicata, l'applicazione apre una finestra di dialogo popup modale (`SettingsPopupFX`) che si sovrappone in modo ordinato alla schermata principale, senza sostituirsi ad essa né occupare invasivamente l'intero schermo. All'interno del pannello è possibile personalizzare:

1. **Modalità Visiva (*Appearance*):** Commutazione immediata tra il tema scuro (*Dark*) e il tema chiaro (*Light*), con aggiornamento dinamico a runtime dei fogli di stile CSS su tutti i componenti visivi.

2. **Famiglia Tipografica (*Font Family*):** Selezione del font di visualizzazione dell'intera interfaccia (es. System, Segoe UI, Roboto, Arial), garantendo una resa ottimale su ogni tipologia di schermo e densità di pixel.

3. **Stile dei Testi (*Bold text*):** Casella di spunta per attivare o disattivare a livello globale il grassetto su etichette, pulsanti e titoli.

4. **Valuta Base Globale (*Base Currency*):** Scelta della divisa di riferimento predefinita dell'applicazione (es. EUR, USD, CHF). Tutti i resoconti patrimoniali, i saldi cumulativi e i grafici utilizzeranno questa valuta come unità di misura globale.

5. **Cronologia Configurazioni (*History*):** Un riquadro dedicato permette di visualizzare la cronologia delle configurazioni salvate nel tempo, memorizzate tramite snapshot storici.

6. **Applicazione e Annullamento:** Premendo il pulsante `"Apply"`, le preferenze vengono applicate all'istante su tutta la GUI e rese persistenti su disco; premendo `"Cancel"` (o chiudendo la finestra), ogni modifica temporanea viene scartata ripristinando la configurazione originaria.

Gestione e Portabilità dei Dati: la Cartella `data/`

Tutte le informazioni create e modificate dall'utente durante l'utilizzo di UniBodget — inclusi i portafogli attivi, i movimenti contabili inseriti, i limiti di budget mensili, le categorie di spesa, le coppie valutarie della watchlist e le preferenze grafiche personalizzate — vengono salvate in file JSON posizionati all'interno della cartella `data/` (o `./data`).

Per un corretto utilizzo quotidiano, l'utente deve tenere a mente le seguenti semplici indicazioni operative:

- **Portabilità e copie di sicurezza (Backup):** Se si desidera fare una copia di riserva dei propri dati finanziari, trasferire il programma su una chiavetta USB o eseguirlo su un computer diverso continuando da dove si era rimasti, **è fondamentale portarsi sempre dietro l'intera cartella `data/`** assieme all'applicazione.

- **Ricerca automatica della cartella:** All'avvio, l'applicazione cerca autonomamente la cartella `data/` nella posizione in cui si trova il programma. Se l'utente dovesse avviare l'eseguibile da un percorso leggermente diverso o spostare i file in una sottocartella, il software è abbastanza intelligente da scansionare in autonomia le cartelle vicine e superiori per rintracciare i propri file di configurazione senza richiedere interventi manuali.

- **Cosa succede se la cartella manca (Ripartenza da zero):** Qualora la cartella `data/` non dovesse essere trovata in nessun percorso adiacente (ad esempio perché è stata rimossa, non è stata copiata assieme al software o il programma è stato avviato in una cartella completamente vuota), UniBodget non andrà in crash: provvederà invece a **creare automaticamente una nuova cartella `data/` vergine**, inizializzandola con i dati di esempio e le impostazioni predefinite di fabbrica. In questa situazione, l'utente si ritroverà l'applicazione pulita e **si ripartirà da zero**, senza vedere lo storico precedentemente memorizzato. Se ciò dovesse accadere accidentalmente, sarà

sufficiente richiudere l'applicazione, ripristinare la cartella `data/` originale accanto al programma e riavviarlo per ritrovare immediatamente tutti i propri movimenti intatti.

Appendice B

Esercitazioni di Laboratorio

Di seguito si riportano i riferimenti delle consegne settimanali relative all'insegnamento di Programmazione ad Oggetti.

Arianna Palmucci (`arianna.palmucci@studio.unibo.it`)

Nota: Per i laboratori da 01 a 06 le consegne settimanali sono consultabili sui relativi repository GitHub; a partire dal Laboratorio 07 le consegne sono state registrate direttamente sui thread del forum su Virtuale:

- **Laboratorio 01:** https://github.com/palmucci4rianna/oop-lab1
- **Laboratorio 02:** https://github.com/palmucci4rianna/oop-lab2
- **Laboratorio 03:** https://github.com/palmucci4rianna/oop-lab3
- **Laboratorio 04:** https://github.com/palmucci4rianna/oop-lab4
- **Laboratorio 05:** https://github.com/palmucci4rianna/oop-lab5
- **Laboratorio 06:** https://github.com/palmucci4rianna/lab06 e https://github.com/palmucci4rianna/lab06-61pt2
- **Laboratorio 07:** https://virtuale.unibo.it/mod/forum/discuss.php?d=207193#p284992
- **Laboratorio 08:** https://virtuale.unibo.it/mod/forum/discuss.php?d=207921#p286196
- **Laboratorio 09:** https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p287253
- **Laboratorio 10:** https://virtuale.unibo.it/mod/forum/discuss.php?d=209589#p288497
- **Laboratorio 11:** https://virtuale.unibo.it/mod/forum/discuss.php?d=210617#p289688
- **Laboratorio 12:**

Daniele Zappalà (daniele.zappala3@studio.unibo.it)

- **Laboratorio 01:**
- **Laboratorio 02:**
- **Laboratorio 03:**
- **Laboratorio 04:**
- **Laboratorio 05:**
- **Laboratorio 06:**
- **Laboratorio 07:**
- **Laboratorio 08:**
- **Laboratorio 09:**
- **Laboratorio 10:**
- **Laboratorio 11:**
- **Laboratorio 12:**

Bibliografia

[1] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995.

[2] Joshua Bloch. *Effective Java (3rd Edition)*. Addison-Wesley Professional, 2018.

[3] OpenJFX Documentation. *Client Application Framework for Java*. https://openjfx.io/

[4] FasterXML. *Jackson JSON Processor Documentation*. https://github.com/FasterXML/jackson

[5] International Organization for Standardization. *ISO 4217 Currency Codes*. https://www.iso.org/iso-4217-currency-codes.html

[6] Y. Shafranovich. *Common Format and MIME Type for Comma-Separated Values (CSV) Files*. RFC 4180, 2005.

[7] Oracle Corporation. *Lesson: Generics (Updated to Java SE 21)*. https://docs.oracle.com/javase/tutorial/java/generics/

[8] Oracle Corporation. *Package java.net.http Specification (Java SE 21)*. https://docs.oracle.com/en/java/javase/21/docs/api/java.net.http/package-summary.html

[9] Oracle Corporation. *Trail: Date Time (Java SE Tutorial, JSR 310)*. https://docs.oracle.com/javase/tutorial/datetime/

[10] OpenJFX Project. *JavaFX CSS Reference Guide*. https://openjfx.io/javadoc/21/javafx.graphics/javafx/scene/doc-files/cssref.html

[11] Alexander Shvets. *Design Patterns in Java*. Refactoring.Guru, https://refactoring.guru/design-patterns/java

[12] Baeldung. *Exploring the Modern Java HTTP Client*. https://www.baeldung.com/java-9-http-client

[13] Frankfurter API. *Open-source Currency Exchange Rates API*. https://api.frankfurter.app/